

CLANKER · SECURITY REVIEW

Clanker Protocol Review.

A security review of Protocol Review.

CLIENT	PROJECT	AUDITORS	AUDIT WINDOW	FINAL REPORT
Clanker	Protocol Review	Guardian	June 29 – July 29, 2026	August 5, 2026

§ 00

Contents.

100 pages — overview, confidence ranking, scope, 74 findings, disclaimer, and about Guardian.

§ 01	Overview	P. 03
§ 02	Confidence Ranking	P. 04
§ 03	Scope & Classifications	P. 05
	Files in Scope	P. 06
§ 04	Findings	P. 08
	Main Review · 45 findings	P. 13
	Remediation Review · 29 findings	P. 68
§ 05	Disclaimer	P. 99
§ 06	About Guardian	P. 100

§ 01

Overview.

Guardian reviewed Protocol Review across 2 review rounds. The final report includes 20 High, 13 Medium, 20 Low, 21 Informational findings.

AUDIT FIRM	Guardian	REPOSITORY	clanker-devco/contracts
CLIENT	Clanker	METHODOLOGY	Smart contract · Manual · Main Review · Remediation Review
PROJECT	Protocol Review	MAIN REVIEW COMMIT	ef734f58e018
PREPARED BY	Guardian	REMEDATION REVIEW COMMIT	90655e9efbf4
AUDIT WINDOW	June 29 – July 29, 2026		
FINAL REPORT	August 5, 2026		

1.1 Confidence Ranking

3 / 5

Moderate Confidence

Guardian assigns a Confidence Ranking of 3 out of 5, combining finding severity, remediation outcome, and the reviewer’s assessment of latent risk. The full scale is on the next page.

1	2	3	4	5
VERY LOW CONFIDENCE	LOW CONFIDENCE	MODERATE CONFIDENCE	HIGH CONFIDENCE	VERY HIGH CONFIDENCE

1.2 Findings at a Glance

SEVERITY	TOTAL	PENDING	ACK	PARTIAL	RESOLVED
Critical	0	0	0	0	0
High	20	0	8	4	8
Medium	13	0	5	2	6
Low	20	0	7	1	12
Informational	21	0	10	1	10
All severities	74	0	30	8	36

§ 02

Confidence Ranking.

Guardian assigns a project-level confidence score using the scale below. The ranking combines finding severity, remediation outcome, and the reviewer's assessment of latent risk.

CONFIDENCE RANKING	DEFINITION AND RECOMMENDATION	RISK PROFILE
5 Very High Confidence	Codebase is mature, clean, and secure. No High or Critical vulnerabilities were found. Follows modern best practices with high test coverage and thoughtful design. Recommendation: code is highly secure at time of audit.	0 High/Critical findings and few Low/Medium severity findings.
4 High Confidence	Code is clean, well-structured, and adheres to best practices. Design patterns are sound and test coverage is strong. Recommendation: suitable for deployment after remediations; consider periodic review with changes.	0-1 High/Critical findings per engagement week and little to no Medium severity findings.
3 Moderate Confidence ASSIGNED	Code is functional but shows gaps in defensive programming or test coverage. Recommendation: remediate the findings and re-review the changes before deployment.	Some High/Critical findings, with various Medium severity findings.
2 Low Confidence	Recurring issues suggest systemic gaps rather than isolated mistakes. Recommendation: remediate, then commission a follow-up review at a frozen commit.	Over 6 High and Critical findings across the engagement.
1 Very Low Confidence	The volume and severity of findings indicate the codebase is not ready for production. Recommendation: substantial rework, then a full re-review.	Over 12 High and Critical findings across the engagement.

§ 03

Scope & Classifications.

Guardian reviewed Protocol Review in `clanker-devco/contracts`. The scope includes 4,713 source lines across 46 files.

3.1 Severity Matrix

	IMP · HIGH	IMP · MED	IMP · LOW
LIK · HIGH	Critical	High	Medium
LIK · MED	High	Medium	Low
LIK · LOW	Medium	Low	Low

IMPACT · HIGH
Significant loss of assets, harm to a group of users, or core protocol functionality is disrupted.

IMPACT · MEDIUM
Small amount of funds can be lost, or ancillary functionality is affected. Users may experience reduced or delayed receipt.

IMPACT · LOW
Unexpected behavior in some functionality that is notable but does not meet the criteria for higher severity.

LIKELIHOOD · HIGH
Possible with reasonable assumptions that mimic on-chain conditions; cost is low compared to amount gained.

LIKELIHOOD · MEDIUM
Possible only in uncommon cases or requires a large amount of capital relative to disruption caused.

LIKELIHOOD · LOW
Unlikely to ever occur in production.

Files in Scope 1/2.

FILE	NSLOC	TOTAL
src/v5/ClankerV5.sol	744	744
src/v5/ClankerCompatibilityRegistry.sol	616	616
src/v5/ClankerDeploymentRegistry.sol	67	67
src/v5/adapters/UniswapV4Adapter.sol	276	276
src/v5/adapters/PancakeInfinityCLAdapter.sol	117	117
src/v5/adapters/BaselineMercuryWrapper.sol	104	104
src/v5/adapters/AmmAdapterModuleBase.sol	73	73
src/v5/adapters/types/AmmAdapterBlobTypes.sol	28	28
src/v5/extensions/ClankerUniv4EthDevBuyV5.sol	80	80
src/v5/extensions/ExtensionModuleBase.sol	57	57
src/v5/extensions/ClankerVaultV5.sol	63	63
src/v5/extensions/ClankerAirdropV5.sol	63	63
src/v5/extensions/ClankerAirdropV5V2.sol	63	63
src/v5/fees/FeeRoutingModuleBase.sol	73	73
src/v5/fees/ClankerFeeRouting.sol	44	44
src/v5/fees/types/FeeRoutingBlobTypes.sol	12	12
src/v5/mechanics/LaunchMechanicModuleBase.sol	73	73
src/v5/mechanics/DefaultLaunchMechanic.sol	65	65
src/v5/mechanics/types/MechanicBlobTypes.sol	15	15
src/v5/token-sources/BaseBTokenSource.sol	197	197
src/v5/token-sources/ClankerTokenSource.sol	136	136
src/v5/token-sources/TokenSourceModuleBase.sol	83	83
src/v5/token-sources/TokenDescriptorRegistry.sol	75	75
src/v5/token-sources/types/TokenSourceBlobTypes.sol	33	33
src/v5/libraries/LaunchValidation.sol	134	134
src/v5/libraries/ClankerV5Constants.sol	129	129
src/v5/libraries/SupplyAccounting.sol	69	69
src/v5/libraries/BaseBTokenInitCallValidation.sol	54	54
src/v5/libraries/ClankerModuleKindLibrary.sol	53	53
src/v5/libraries/UniswapV4PoolKeyValidation.sol	51	51
src/v5/libraries/UniswapV4LaunchIndexing.sol	38	38
src/v5/types/ClankerV5Types.sol	161	161
src/v5/interfaces/IClankerCompatibilityRegistry.sol	234	234
src/v5/interfaces/IClankerV5.sol	222	222
src/v5/interfaces/ITokenDescriptorRegistry.sol	65	65
src/v5/interfaces/IClankerV5LaunchFactory.sol	56	56

Files in Scope 2/2.

FILE	NSLOC	TOTAL
src/v5/interfaces/IClankerAmmAdapter.sol	41	41
src/v5/interfaces/IClankerTokenSource.sol	37	37
src/v5/interfaces/IClankerDeploymentRegistry.sol	37	37
src/v5/interfaces/IPancakeInfinityCLLauncher.sol	35	35
src/v5/interfaces/IClankerExtensionV5.sol	31	31
src/v5/interfaces/IClankerFeeRouting.sol	25	25
src/v5/interfaces/IBaselineMercuryLauncher.sol	25	25
src/v5/interfaces/IClankerModule.sol	24	24
src/v5/interfaces/IClankerLaunchMechanic.sol	24	24
src/v5/interfaces/IClankerUniswapV4MevFinalizer.sol	11	11

§ 04

Findings.

Grouped by round. Visible IDs reset by severity within each round, so a finding is named the same way in the report as it was in the channel it was written in.

ID	TITLE	CATEGORY	SEVERITY	STATUS	PAGE
MAIN REVIEW					
H-01	Forced ETH Transfer Bricks All Launches	DoS	HIGH	RESOLVED	p. 13
H-02	Hidden Genesis Mint Bypasses Supply Conservation	Validation	HIGH	RESOLVED	p. 15
H-03	Baseline Launches With Modular Fee Can Be DoS'ed	DoS	HIGH	RESOLVED	p. 17
H-04	Adapter Donations Block Liquidity Additions	DoS	HIGH	ACKNOWLEDGED	p. 19
H-05	Tick Gap Bypasses Launch FDV Guard	Validation	HIGH	PARTIALLY RESOLVED	p. 21
H-06	Extension Refund Reentrancy Drains Core Fees	Reentrancy	HIGH	RESOLVED	p. 23
H-07	Same Pool Liquidity Additions Always Fail	Compatibility	HIGH	PARTIALLY RESOLVED	p. 26
H-08	All Passive Baseline Fees Can Be Stolen	Access Control	HIGH	RESOLVED	p. 27
H-09	FDV Guard Bypass Via Mining Clanker as currency1	Logical Error	HIGH	RESOLVED	p. 28
H-10	Stolen Protocol Fees Due To Arbitrary Recipient	Validation	HIGH	PARTIALLY RESOLVED	p. 30
H-11	Uniswap Supplemental Liquidity Always Reverts	DoS	HIGH	RESOLVED	p. 31
H-12	Unchecked lockerData Allows Malicious Fee Module	Validation	HIGH	RESOLVED	p. 32

Findings continued.

ID	TITLE	CATEGORY	SEVERITY	STATUS	PAGE
H-13	collectRewardsWithoutUnlock Lets Fees Stolen	Access Control	HIGH	PARTIALLY RESOLVED	p. 33
M-01	Venueld Collision With Different Pancake Fees	Logical Error	MEDIUM	RESOLVED	p. 34
M-02	DevBuy ETH Can Be Stranded In UniversalRouter	Logical Error	MEDIUM	PARTIALLY RESOLVED	p. 35
M-03	Admin Rotation Is Not Recorded	Logical Error	MEDIUM	RESOLVED	p. 36
M-04	FDV Guard Does Not Validate Denomination Token	Validation	MEDIUM	RESOLVED	p. 37
M-05	Raw Fee Units Skew Thresholds	Logical Error	MEDIUM	RESOLVED	p. 39
M-06	Lost LP Fees When v4-parity With Pancake	Configuration	MEDIUM	RESOLVED	p. 40
M-07	Supplemental Pools Can Break Fees And Swaps	Validation	MEDIUM	RESOLVED	p. 41
M-08	Pancake Residuals Bypass Consumption Check	Rounding	MEDIUM	PARTIALLY RESOLVED	p. 43
L-01	setCore Missing Access Control	Access Control	LOW	RESOLVED	p. 44
L-02	Permissionless createToken Enables Launch Grief	Access Control	LOW	RESOLVED	p. 45
L-03	Caller launchId Enables DoS And Collision	DoS	LOW	RESOLVED	p. 46
L-04	B-20 Descriptor Cannot Derive Address	Configuration	LOW	RESOLVED	p. 47
L-05	Fee Token Can Mismatch Venue Assets	Validation	LOW	RESOLVED	p. 48
L-06	Public Adapters Spoof Launch Events	Access Control	LOW	PARTIALLY RESOLVED	p. 49
L-07	Relayed Launch Extension Refund Mismatch	Logical Error	LOW	RESOLVED	p. 50

Findings continued.

ID	TITLE	CATEGORY	SEVERITY	STATUS	PAGE
L-08	Passive Fees Can Be Misattributed	Logical Error	LOW	RESOLVED	p. 51
L-09	Uniswap Adapter Underreports Capabilities	Compatibility	LOW	RESOLVED	p. 52
L-10	BToken Descriptors Can Be False	Validation	LOW	RESOLVED	p. 53
L-11	BToken Capabilities Are Underreported	Compatibility	LOW	RESOLVED	p. 54
L-12	Airdrop V2 Allows Admin Root Replacement	Validation	LOW	RESOLVED	p. 55
L-13	Passive Mode Accepted For Active Venues	Validation	LOW	RESOLVED	p. 56
I-01	Asset Multiplier Not Shown in Capabilities	Informational	INFORMATIONAL	RESOLVED	p. 57
I-02	Zero totalSupply Diverges Between Token Sources	Documentation	INFORMATIONAL	RESOLVED	p. 58
I-03	Baseline Launches Require Zero Extensions	Validation	INFORMATIONAL	RESOLVED	p. 59
I-04	Warning Regarding B20 Transfer Policies	Trust Assumptions	INFORMATIONAL	RESOLVED	p. 60
I-05	Misleading InvalidLockerRewards Error Usage	Error	INFORMATIONAL	RESOLVED	p. 61
I-06	Supplemental MEV Reinitialization Risk	Warning	INFORMATIONAL	RESOLVED	p. 62
I-07	Pancake Allows Arbitrary Static LP Fee	Validation	INFORMATIONAL	RESOLVED	p. 63
I-08	Baseline Fee Sink Remains Mutable	Warning	INFORMATIONAL	RESOLVED	p. 64
I-09	Mercury Requires Full Supply At Launch	Informational	INFORMATIONAL	RESOLVED	p. 65
I-10	Supplemental Liquidity Re-binds Revoked Fee Module	Informational	INFORMATIONAL	PARTIALLY RESOLVED	p. 66

Findings continued.

ID	TITLE	CATEGORY	SEVERITY	STATUS	PAGE
I-11	Pancake Default Params Ignore Decimals	Informational	INFORMATIONAL	RESOLVED	p. 67
REMEDATION REVIEW					
H-01	Supplemental Liquidity Can Still Be DoS'ed	DoS	HIGH	ACKNOWLEDGED	p. 68
H-02	Launches With ETH Buy Can Easily DoS'ed	DoS	HIGH	ACKNOWLEDGED	p. 69
H-03	Broken Supplemental Liquidity Due To venueId	DoS	HIGH	ACKNOWLEDGED	p. 70
H-04	Min FDV Bypass Via 1 BPS Allocation	Logical Error	HIGH	ACKNOWLEDGED	p. 71
H-05	Hook-Only Fee Collection Still Allows Stolen Fee	Access Control	HIGH	ACKNOWLEDGED	p. 72
H-06	Pancake Launches Revert On Rounding Dust	Compatibility	HIGH	ACKNOWLEDGED	p. 74
H-07	Shared Hook Enables Cross-Pool Fee Theft	Logical Error	HIGH	ACKNOWLEDGED	p. 76
M-01	Crosschain FDV Uses Local Supply	Logical Error	MEDIUM	ACKNOWLEDGED	p. 77
M-02	One Token-Restricted Beneficiary Blocks Fees	Logical Error	MEDIUM	ACKNOWLEDGED	p. 78
M-03	Pancake Defaults Depend On Token Order	Logical Error	MEDIUM	ACKNOWLEDGED	p. 79
M-04	Baseline Uses Local Supply As Full Supply	Compatibility	MEDIUM	ACKNOWLEDGED	p. 80
M-05	Zero Protocol Numerators Disable Fees	Logical Error	MEDIUM	ACKNOWLEDGED	p. 81
L-01	Mechanic Context Uses Raw Launch Id	Compatibility	LOW	ACKNOWLEDGED	p. 82
L-02	Revoked Fee Module Blocks Supplemental Adds	DoS	LOW	ACKNOWLEDGED	p. 83

Findings continued.

ID	TITLE	CATEGORY	SEVERITY	STATUS	PAGE
L-03	Missing onlyLaunchFactory When Adding Liquidity	Access Control	LOW	ACKNOWLEDGED	p. 84
L-04	Descriptor Records Only Local Supply	Logical Error	LOW	ACKNOWLEDGED	p. 85
L-05	Synced Pool Squatting Across Chains	Frontrunning	LOW	ACKNOWLEDGED	p. 86
L-06	Reauthorization Revives Stale Forwarders	Logical Error	LOW	ACKNOWLEDGED	p. 87
L-07	Getter Hides Paired-Token Pending Fees	Unexpected Behavior	LOW	ACKNOWLEDGED	p. 88
I-01	Note Regarding Uniswap Supplemental Liquidity	Informational	INFORMATIONAL	ACKNOWLEDGED	p. 89
I-02	Refunds Require Payable Callers	Documentation	INFORMATIONAL	ACKNOWLEDGED	p. 90
I-03	Document Dev-Buy Recipient Also Receives Refunds	Documentation	INFORMATIONAL	ACKNOWLEDGED	p. 91
I-04	Registry Admin Update Event Is Not Emitted	Events	INFORMATIONAL	ACKNOWLEDGED	p. 92
I-05	Supplemental CL Ranges Must Match Price	Documentation	INFORMATIONAL	ACKNOWLEDGED	p. 93
I-06	Baseline Deploy Docs Use Old Address	Documentation	INFORMATIONAL	ACKNOWLEDGED	p. 94
I-07	Warning Regarding Multi Chains	Warning	INFORMATIONAL	ACKNOWLEDGED	p. 95
I-08	Unbound Bridge LaunchId	Validation	INFORMATIONAL	ACKNOWLEDGED	p. 96
I-09	Passive Sinks Assume Standard ERC20	Informational	INFORMATIONAL	ACKNOWLEDGED	p. 97
I-10	Misleading Comment In Fee Module	Informational	INFORMATIONAL	ACKNOWLEDGED	p. 98

HIGH **H-01**

RESOLVED

Forced ETH Transfer Bricks All Launches

CATEGORY	DoS
LOCATION	https://github.com/GuardianOrg/contracts-team1-1782421575052/blob/ef734f58e018a0eee57d78d3f5a0d8f01c082edc/src/v5/ClankerV5.sol#L235-L237
REVIEW	Main Review

DESCRIPTION

`deployLaunch` finalizes with a strict zero-balance check:

```
if (address(this).balance != 0) {  
    revert MsgValueMismatch();  
}
```

This assumes core holds no ETH beyond the `msg.value` that extensions consume during the launch.

Although there is no receive/fallback mechanism, anyone can force ETH into core with

`selfdestruct(core)`, which `EIP-6780` still allows if the attack contract is created and destroyed in the same transaction.

From that point every `deployLaunch` reverts at this check, since `address(this).balance > 0` always holds, as the deployment function only consumes `msg.value = requiredMsgValue`.

RECOMMENDATION

This check already guarantees the user has provided sufficient ETH:

```
if (msg.value != requiredMsgValue) {  
    revert MsgValueMismatch();  
}
```

Therefore, consider refactoring the `if (address(this).balance != 0)` check to refund excess ETH by tracking before and after amounts:

```
// at entry, before extensions run  
uint256 ethBaseline = address(this).balance - msg.value;  
// ...  
// at exit  
uint256 ethBalance = address(this).balance;  
if (ethBalance < ethBaseline) {  
    revert MsgValueMismatch(); // can't consume more than supplied  
}
```

H-01 Forced ETH Transfer Bricks All Launches

CONTINUED

```
uint256 launchEthRemaining = ethBalance - ethBaseline;
if (launchEthRemaining != 0) {
    // Refund unspent launch ETH to creator or relayer
    (bool refunded,) = payable(msg.sender).call{value: launchEthRemaining}("");
    if (!refunded) revert MsgValueMismatch();
}
```

Consider if you want to refund to `msg.sender` or `envelope.creator`.

— RESOLUTION

Status: Resolved.

HIGH **H-02**

RESOLVED

Hidden Genesis Mint Bypasses Supply Conservation

CATEGORY Validation

LOCATION <https://github.com/GuardianOrg/contracts-team1-1782421575052/blob/ef734f58e018a0eee57d78d3f5a0d8f01c082edc/src/v5/token-sources/BaseTokenSource.sol#L93>, <https://github.com/GuardianOrg/contracts-team1-1782421575052/blob/ef734f58e018a0eee57d78d3f5a0d8f01c082edc/src/v5/token-sources/BaseTokenSource.sol#L126>, <https://github.com/GuardianOrg/contracts-team1-1782421575052/blob/ef734f58e018a0eee57d78d3f5a0d8f01c082edc/src/v5/token-sources/BaseTokenSource.sol#L138-L140>

REVIEW Main Review

DESCRIPTION

`BaseTokenSource.createToken` treats `balanceOf(source) == config.totalSupply` as its only supply-conservation check and never validates the token's real `totalSupply()`. The B-20 factory mints nothing itself, as the entire supply is inputted by caller-controlled `mint/batchMint` `initCalls`. Because `BaseTokenInitCallValidation` gates selectors (and not arguments), a `mint(attacker, 5x) initCall` passes validation: the mint selector is allowlisted and the recipient and amount are never checked.

A creator adds one extra mint alongside the required one:

```
initCalls = [ mint(source, TOTAL_SUPPLY), // satisfies balanceOf(source) == totalSupply
mint(attacker, 5 * TOTAL_SUPPLY) ]; // hidden, never accounted
```

The launch reports `totalLaunchSupply == TOTAL_SUPPLY` and seeds liquidity as if that is 100% of supply, while the token's real `totalSupply()` is 6x, with 5x already sitting in the creator's wallet, invisible to the launch and to v5's supply-conservation invariant. The creator can then dump the hidden supply into the freshly seeded pool.

This does not apply to `ClankerTokenSource`, since it mints a fixed supply in the token constructor, so `balanceOf(source) == totalSupply` implies `token.totalSupply() == totalSupply`.

RECOMMENDATION

In `BaseTokenSource.createToken`, assert `IERC20(token).totalSupply() == config.totalSupply` in addition to the existing source-balance check. The balance check enforces that the accounted supply landed with the source, while the total-supply check closes the mint-elsewhere hole, since any extra genesis mint inflates `totalSupply()`. Also consider validating `BaseTokenInitCallValidation` `mint/batchMint` arguments, restricting recipients to the source or disallowing genesis mints outside the accounted supply.

H-02 Hidden Genesis Mint Bypasses Supply Conservation

CONTINUED

PROOF OF CONCEPT <https://github.com/GuardianOrg/contracts-team1-1782421575052/tree/poc/b20-hidden-premint>

— RESOLUTION

Status: Resolved.

HIGH **H-03**

RESOLVED

Baseline Launches With Modular Fee Can Be DoS'ed

CATEGORY	DoS
LOCATION	https://github.com/GuardianOrg/contracts-team1-1782421575052/blob/ef734f58e018a0eee57d78d3f5a0d8f01c082edc/src/v5/fees/ClankerVolumeThresholdFee.sol#L65
REVIEW	Main Review

— DESCRIPTION

The intended v5 fee-routing path is modular fee routing, where launches route fees through a fee module rather than relying only on the legacy/v4-parity path. In the current implementation, this means `ClankerVolumeThresholdFee`, which supports both active and passive fee collection. Uniswap/Pancake-style fee flows are active, while Baseline Mercury fee flows are always passive because Mercury sends reserve-token fees directly to the configured fee recipient.

Passive fee collection is guarded by `_passiveFeeTokenVenues`, but this mapping is keyed only by fee token and not by `launchId` or venue instance. As a result, the shared `ClankerVolumeThresholdFee` can only initialize one passive Baseline fee policy per reserve token. Once WETH, USDC, or another reserve token has been used by any passive Baseline launch, subsequent Baseline launches using modular fee routing and that same reserve token revert during fee policy initialization.

This can happen naturally as normal launches reuse common reserve assets, or it can be forced by an attacker. Because launches are permissionless once the adapter and modular fee path are enabled, an attacker can cheaply launch arbitrary tokens once per common reserve token, permanently occupying those reserve-token slots in the shared fee module and DoS'ing later modular-fee Baseline launches for those reserves.

— RECOMMENDATION

Route Baseline passive fees to launch-specific fee sink contracts that receive Mercury fees for a single launch/reserve and then forward attributed amounts into the shared fee module, or alternatively deploy isolated fee module instances for passive Baseline launches.

PROOF OF CONCEPT <https://github.com/GuardianOrg/contracts-team1-1782421575052/pull/3>

H-03 Baseline Launches With Modular Fee Can Be DoS'ed

CONTINUED

— RESOLUTION

Status: Resolved.

HIGH **H-04**

ACKNOWLEDGED

Adapter Donations Block Liquidity Additions

CATEGORY	DoS
LOCATION	https://github.com/GuardianOrg/contracts-team1-1782421575052/blob/ef734f58e018a0eee57d78d3f5a0d8f01c082edc/src/v5/adapters/BaselineMercuryWrapper.sol#L97-L100 , https://github.com/GuardianOrg/contracts-team1-1782421575052/blob/ef734f58e018a0eee57d78d3f5a0d8f01c082edc/src/v5/adapters/PancakeInfinityCLAdapter.sol#L124-L126
REVIEW	Main Review

— DESCRIPTION

After a successful launch, any holder of the launch token can transfer a small amount of that token directly to the Pancake or Baseline adapter. Later, when the token admin attempts to add supplemental liquidity through `addLiquidityPosition`, core transfers the intended `tokenAmount` to the adapter, but the adapter's existing donated balance interferes with its accounting.

For Pancake, the launcher consumes exactly the assigned `tokenAmount`, leaving the donated dust in the adapter. The adapter then checks that its launch-token balance is zero and reverts with `LauncherDidNotConsumeTokens`. For Baseline, the adapter checks before calling Mercury that its total launch-token balance equals the assigned `tokenAmount`; because the donated amount makes the balance `tokenAmount + donation`, it reverts with `ExceedsAssignedTokenAmount`.

This creates a repeatable grieving vector against supplemental liquidity additions for Pancake venues. Admin asset recovery can clear the adapter balance, but an attacker can donate again after recovery, forcing operational intervention each time the token admin wants to add liquidity. Additionally, initial launch validation does not allow the same adapter to be used more than once in the launch envelope, so a token owner who wants to add another liquidity position through the same venue/adapter after launch must use `addLiquidityPosition`.

This does not affect Uniswap v4 in the same way as its adapter approves and places only the assigned `tokenAmount` without requiring the adapter's pre- or post-call token balance to exactly match that amount.

— RECOMMENDATION

Make adapter accounting delta-based or assigned-amount-based instead of absolute-balance-based, so pre-existing donated balances are ignored or swept separately while the adapter still verifies that only the intended `tokenAmount` is consumed for the liquidity operation

H-04 Adapter Donations Block Liquidity Additions

CONTINUED

PROOF OF CONCEPT <https://github.com/GuardianOrg/contracts-team1-1782421575052/pull/4>

— RESOLUTION

Status: Acknowledged.

HIGH **H-05**

PARTIALLY RESOLVED

Tick Gap Bypasses Launch FDV Guard

CATEGORY Validation

LOCATION <https://github.com/GuardianOrg/contracts-team1-1782421575052/blob/ef734f58e018a0eee57d78d3f5a0d8f01c082edc/src/v5/ClankerV5.sol#L203>, <https://github.com/GuardianOrg/contracts-team1-1782421575052/blob/ef734f58e018a0eee57d78d3f5a0d8f01c082edc/src/v5/adapters/UniswapV4Adapter.sol#L238-L257>, <https://github.com/GuardianOrg/contracts-team1-1782421575052/blob/ef734f58e018a0eee57d78d3f5a0d8f01c082edc/src/v5/adapters/UniswapV4Adapter.sol#L129-L141>, <https://github.com/GuardianOrg/contracts-team1-1782421575052/blob/ef734f58e018a0eee57d78d3f5a0d8f01c082edc/src/v5/adapters/UniswapV4Adapter.sol#L183-L187>

REVIEW Main Review

DESCRIPTION

The launch valuation guard relies on the adapter's `quoteLaunchFdvPaired()` result to enforce max-FDV limits. In the Uniswap v4 adapter, the quoted FDV is computed from the pool's initialized starting tick.

```
int24 startingTick = token0IsClanker
? config.poolConfig.tickIfToken0IsClanker
: -config.poolConfig.tickIfToken0IsClanker;
uint160 sqrtPriceX96 = TickMath.getSqrtPriceAtTick(startingTick);
fdvPaired = LaunchValuationMath.fdvPairedFromSqrtPriceX96(
context.supplyAllocation.totalLaunchSupply,
sqrtPriceX96,
token0IsClanker
);
```

However, liquidity is placed separately through the locker configuration `IClankerV5LaunchFactory(LAUNCH_FACTORY).factoryPlaceV4Liquidity`.

Both UniswapV4 and PancakeInfinity adapter quotes price FDV at the init spot.

UniswapV4Adapter.quoteLaunchFdvPaired computes it from `tickIfToken0IsClanker`, and PancakeInfinityCLAdapter.quoteLaunchFdvPaired from `params.sqrtPriceX9` (both via `LaunchValuationMath`). Meanwhile the liquidity is single-sided and minted in `[tickLower, tickUpper]`, and the only constraint tying it to spot is the locker's `tickLower[i] >= startingTick` check, which prevents liquidity below spot but does not require `min (tickLower) == startingTick`.

Therefore, for the same pool, launches can accept a malicious config such as `poolConfig.tickIfToken0IsClanker = -230400; // very low starting tick, lockerConfig.tickLower[0] = -30000; // much higher first liquidity tick, lockerConfig.positionBps[0] = 10_000; // all liquidity here`.

The adapter attempts to validate the tick arrays via `UniswapV4Adapter._validateConfig`, but it does not require the lowest liquidity tick to equal the initialized starting tick. As a result, a creator can initialize the pool at a low starting tick to pass the max-FDV

H-05 Tick Gap Bypasses Launch FDV Guard

CONTINUED

guard, then place all launch liquidity at a much higher tickLower. This creates a liquidity-free gap between the guarded spot price and the first executable liquidity price.

The attached PoCs confirm this, as a launch initialized at startingTick = -230400 passed valuation using an FDV of approximately 9.87 WETH, while all liquidity was placed near tickLower = -30000. A real WETH to token swap skipped the empty gap and filled near -29999, giving an effective entry FDV of approximately 4.98e9 WETH. This demonstrates that buyers can enter at a valuation far above the value checked by the guard.

This defeats the purpose of max-FDV guarded launches. The launch appears compliant because the guard measures the initialized spot price, but users actually trade at the first executable liquidity price.

— RECOMMENDATION

Compute guarded FDV at the first executable liquidity price instead of the initialized spot. For Uniswap launches, use the minimum tickLower across launch liquidity positions, or require `min(tickLower) == startingTick`. For Pancake launches, use the lowest liquidity tick from the configured range, or require the initial sqrtPriceX96 to match the first liquidity tick.

PROOF OF CONCEPT <https://github.com/GuardianOrg/contracts-team1-1782421575052/tree/poc/uniswap-guard-bypass>

— RESOLUTION

Status: Partially resolved.

HIGH **H-06**

RESOLVED

Extension Refund Reentrancy Drains Core Fees

CATEGORY	Reentrancy
LOCATION	https://github.com/GuardianOrg/contracts-team1-1782421575052/blob/ef734f58e018a0eee57d78d3f5a0d8f01c082edc/src/v5/ClankerV5.sol#L918 , https://github.com/GuardianOrg/contracts-team1-1782421575052/blob/ef734f58e018a0eee57d78d3f5a0d8f01c082edc/src/v5/ClankerV5.sol#L733
REVIEW	Main Review

DESCRIPTION

ClankerV5._executeExtensionAllocations() temporarily authorizes the active extension by setting `_activeExtensionModule`. This flag is intended to allow only the currently executing extension to call sensitive factory helpers such as `factoryTriggerV4Extension()`.

However, in the failed extension path, `_activeExtensionModule` remains set while the contract executes a refund to `tokenAdmin` via an external call:

```
_activeExtensionModule = extensionModule;
try IClankerExtensionV5(extensionModule)
.receiveLaunchAllocation{value: extensionMsgValue}(
context,
envelope.extensionAllocations[i],
extensionSupply
)
{
...
} catch (bytes memory reason) {
...
(bool refunded,) =
payable(context.tokenAdmin).call{value: extensionMsgValue}("");
if (!refunded) {
revert MsgValueMismatch();
}
...
}
_activeExtensionModule = address(0);
```

Because `tokenAdmin` is creator-controlled, an attacker can set it to a contract and reenter during the ETH refund. At that point `_activeExtensionModule` is still set, so the attacker can route through a legitimate registered extension wrapper and reach `factoryTriggerV4Extension()`.

H-06 Extension Refund Reentrancy Drains Core Fees

CONTINUED

That function is gated only by `onlyActiveExtensionModule` and then approves an arbitrary token from the core to the supplied extension path:

```
function factoryTriggerV4Extension(
    address extension,
    IClanker.DeploymentConfig calldata deploymentConfig,
    PoolKey calldata poolKey,
    address token,
    uint256 extensionSupply,
    uint256 extensionIndex
) external payable onlyActiveExtensionModule {
    IERC20 tokenContract = IERC20(token);
    SafeERC20.forceApprove(tokenContract, extension, extensionSupply);
    IClankerExtension(extension).receiveTokens{value: msg.value}(
        deploymentConfig,
        poolKey,
        token,
        extensionSupply,
        extensionIndex
    );
    SafeERC20.forceApprove(tokenContract, extension, 0);
}
```

This allows the attacker to make ClankerV5 approve and transfer foreign ERC20 balances held by the core. The launch can still finalize because the final supply conservation check only verifies the current launch token balance, not unrelated ERC20 balances.

The allows direct theft of accumulated hook protocol fees held by the core, as the `claimTeamFees` natspec states "Claims accumulated hook protocol fees held by core. Distinct from stray recovery; sweeps the full token balance as protocol fees."

Any creator can trigger the issue by configuring a failed extension allocation with a non-reverting failure policy and a refund to an attacker-controlled tokenAdmin. The exploit does not require registering a malicious extension, since a legitimate registered wrapper can be used as the reentry proxy.

— RECOMMENDATION

Clear `_activeExtensionModule` before performing any external refund or token transfer in the failed extension path:

H-06 Extension Refund Reentrancy Drains Core Fees

CONTINUED

```
catch (bytes memory reason) {
    _activeExtensionModule = address(0); //@audit reset here
    ...
    (bool refunded,) =
    payable(context.tokenAdmin).call{value: extensionMsgValue}("");
    if (!refunded) {
        revert MsgValueMismatch();
    }
    ...
}
```

PROOF OF CONCEPT <https://github.com/GuardianOrg/contracts-team1-1782421575052/tree/poc/extension-refund-reentrancy>

— RESOLUTION

Status: Resolved.

HIGH **H-07**

PARTIALLY RESOLVED

Same Pool Liquidity Additions Always Fail

CATEGORY Compatibility

LOCATION <https://github.com/GuardianOrg/contracts-team1-1782421575052/blob/ef734f58e018a0eee57d78d3f5a0d8f01c082edc/src/v5/adapters/BaselineMercuryWrapper.sol#L118>, <https://github.com/GuardianOrg/contracts-team1-1782421575052/blob/ef734f58e018a0eee57d78d3f5a0d8f01c082edc/src/v5/launchers/BaselineMercuryClankerLauncher.sol#L74>, <https://github.com/GuardianOrg/contracts-team1-1782421575052/blob/ef734f58e018a0eee57d78d3f5a0d8f01c082edc/src/v5/launchers/PancakeInfinityCLClankerLauncher.sol#L161>

REVIEW Main Review

DESCRIPTION

`ClankerV5.addLiquidityPosition` is exposed as a post-launch path for appending supplemental liquidity positions, but it calls the same `placeLiquidity` entry point used during the initial deploy flow on all adapters. The production adapters then execute their pool-creation logic again instead of placing liquidity into an already initialized pool.

As a result, same-pool supplemental additions revert across the current production paths. Uniswap v4 tries pool initialization before locker placement, Pancake Infinity CL calls `CL_POOL_MANAGER.initialize(...)`, and Baseline Mercury calls `createPoolFromInvariant(...)`; each path rejects an already initialized pool. This prevents token admins from adding a new LP position in the same pool, including normal use cases such as adding liquidity in a different tick range.

As a result, supplemental liquidity cannot be added to the original launch pool. Callers may still create additional positions by changing pool-key parameters such as the fee tier or paired token, but that creates a different pool rather than adding liquidity to the launch pool and fragments liquidity.

RECOMMENDATION

Do not reuse the same `placeLiquidity` path for supplemental liquidity as that path always attempts to initialize or create the pool. Implement dedicated supplemental-liquidity logic for each AMM that first checks whether the target pool is already initialized, then either adds a new position to the existing pool or initializes the pool before placing the new position.

PROOF OF CONCEPT <https://github.com/GuardianOrg/contracts-team1-1782421575052/pull/5>

RESOLUTION

Status: Partially resolved.

HIGH **H-08**

RESOLVED

All Passive Baseline Fees Can Be Stolen

CATEGORY Access Control

LOCATION <https://github.com/GuardianOrg/contracts-team1-1782421575052/blob/ef734f58e018a0eee57d78d3f5a0d8f01c082edc/src/v5/fees/ClankerVolumeThresholdFee.sol#L194-L206>

REVIEW Main Review

DESCRIPTION

`ClankerVolumeThresholdFee.receiveFeesForDepositor` only checks that the supplied depositor address is authorized for the launch and venue. It does not require `msg.sender` to be that depositor or an approved forwarder. Because authorized depositor addresses are public, any caller can pass a valid depositor and make the fee module account an existing token balance to that venue.

This allows passive Baseline fees to be stolen atomically. Baseline Mercury `claimPoolFees` is permissionless and transfers accrued reserve fees to the v5 fee module. An attacker can call `claimPoolFees` for a Baseline market and then immediately call `receiveFeesForDepositor` for their own active venue, causing the newly transferred reserve fees to be accounted to the attacker's venue. The attacker can then call `distributeFees` for their venue and receive the passive fees through their configured recipients.

Additionally, because active venues with `feeToken == address(0)` accept any ERC20, the same missing caller authentication lets an attacker credit arbitrary worthless token balances as fee revenue, prematurely crossing the threshold or pushing `cumulativeRevenue` near `type(uint256).max` and causing future fee accounting to revert.

RECOMMENDATION

Require `receiveFeesForDepositor` to authenticate `msg.sender` as the depositor or an approved forwarder, and prevent active fee accounting from consuming passive fee balances belonging to other venues.

PROOF OF CONCEPT <https://github.com/GuardianOrg/contracts-team1-1782421575052/pull/6>

RESOLUTION

Status: Resolved.

HIGH **H-09**

RESOLVED

FDV Guard Bypass Via Mining Clanker as currency1

CATEGORY	Logical Error
LOCATION	https://github.com/GuardianOrg/contracts-team1-1782421575052/blob/ef734f58e018a0eee57d78d3f5a0d8f01c082edc/src/v5/launchers/PancakeInfinityCLClankerLauncher.sol#L176-L201
REVIEW	Main Review

DESCRIPTION

Pancake Infinity CL valuation uses the decoded `sqrtPriceX96` from `launchParams`, or derives it from the user-provided `tickLower` when `sqrtPriceX96 == 0`. The production launcher then initializes the pool at that same unmirrored price. However, when the Clanker token sorts as `currency1`, the launcher mirrors only the LP tick range by returning `(-tickUpper, -tickLower)` and does not mirror the initialization price used for valuation.

A malicious creator can intentionally mine/grind the token-source salt so the Clanker token sorts after the paired token, making Clanker `currency1`. They can then provide launch params that pass valuation at the unmirrored price while liquidity is actually deposited in the mirrored range. For example, valuation can be approved at tick +30000, while the real LP range is [-60000, -30000]; the first executable buy-side liquidity sits near -30000, which is materially more expensive (~400x) than the guarded valuation point.

This bypasses valuation-guarded Pancake launches. The launch passes valuation and finalizes as compliant, but executable liquidity sits at an attacker-favorable price far from the approved valuation.

Additionally, this is not limited to valuation-guarded launches: under the default mechanic, Pancake `token1` launches initialize the pool at the unmirrored price while minting liquidity in the mirrored range, causing incorrect pool initialization even without any valuation check.

RECOMMENDATION

Mirror the valuation/init price as well as the liquidity range so both are checked and executed in the same tick space when Clanker is `currency1`. Also, consider enforcing that the guarded price matches the first executable liquidity boundary.

PROOF OF CONCEPT <https://github.com/GuardianOrg/contracts-team1-1782421575052/pull/7>

H-09 FDV Guard Bypass Via Mining Clanker as currency1

CONTINUED

— RESOLUTION

Status: Resolved.

HIGH **H-10**

PARTIALLY RESOLVED

Stolen Protocol Fees Due To Arbitrary Recipient

CATEGORY Validation

LOCATION <https://github.com/GuardianOrg/contracts-team1-1782421575052/blob/ef734f58e018a0eee57d78d3f5a0d8f01c082edc/src/v5/fees/ClankerVolumeThresholdFee.sol#L129>

REVIEW Main Review

DESCRIPTION

`ClankerVolumeThresholdFee` lets launchers provide `protocolRecipient` directly in `feeModuleData`. During fee policy initialization the module stores that address as-is, and validation only requires it to be nonzero.

In the default/v4-parity route, protocol trading fees accrue to the core/factory and are later claimed to the team fee recipient. Contrary to this, a launcher can route the protocol share of modular fee revenue to themselves instead of the Clanker protocol treasury.

This conflicts with the documented economic model where protocol and creator economics come from trading fees after launch. Unless arbitrary protocol-recipient substitution is explicitly intended, modular fee routing lets external launchers bypass Clanker's trading-fee revenue.

RECOMMENDATION

Derive `protocolRecipient` from protocol-controlled configuration, or validate that the caller-provided value equals the approved Clanker protocol fee recipient.

Additionally, if zero protocol fees are not explicitly intended, also enforce protocol-governed minimum values for `initialProtocolNumerator` and `postThresholdProtocolNumerator` to prevent revenue loss.

RESOLUTION

Status: Partially resolved.

HIGH **H-11**

RESOLVED

Uniswap Supplemental Liquidity Always Reverts

CATEGORY DoS

LOCATION <https://github.com/GuardianOrg/contracts-team1-1782421575052/blob/ef734f58e018a0eee57d78d3f5a0d8f01c082edc/src/v5/adapters/UniswapV4Adapter.sol#L184-L186>, <https://github.com/GuardianOrg/contracts-team1-1782421575052/blob/ef734f58e018a0eee57d78d3f5a0d8f01c082edc/src/v5/ClankerV5.sol#L712>

REVIEW Main Review

DESCRIPTION

Uniswap v4 launches require locker reward configuration in practice because the downstream production locker rejects empty reward recipients and stores reward state for each launched token. During the initial launch, `placeLiquidity` function in locker records `_tokenRewards[token]` for the launched token and uses that token-level state to manage the locker's LP reward accounting.

`ClankerV5.addLiquidityPosition` is intended to support supplemental liquidity for the same launched token, but the Uniswap adapter routes supplemental additions through the same `placeLiquidity` deployment path. When the same locker is used again, the locker sees that `_tokenRewards[token]` already exists and reverts with `TokenAlreadyHasRewards`. As a result, Uniswap supplemental liquidity is permanently unusable for normal production launches using the standard locker.

Noting that this is separate from the previous H-07 issue where same pool liquidity additions revert with `PoolAlreadyInitialized`. Even if the token admin avoids the original pool and targets a fresh Uniswap pool, the supplemental add still fails because the locker state.

RECOMMENDATION

Implement dedicated supplemental liquidity flows instead of reusing launch-time `placeLiquidity` functions. For Uniswap, this likely requires locker-level support for appending positions or a separate supplemental custody path, because the current production locker cannot cleanly support same-token supplemental additions without implementation changes.

PROOF OF CONCEPT <https://github.com/GuardianOrg/contracts-team1-1782421575052/pull/8>

RESOLUTION

Status: Resolved.

HIGH **H-12**

RESOLVED

Unchecked lockerData Allows Malicious Fee Module

CATEGORY	Validation
LOCATION	https://github.com/GuardianOrg/contracts-team1-1782421575052/blob/ef734f58e018a0eee57d78d3f5a0d8f01c082edc/src/v5/adapters/UniswapV4Adapter.sol#L154-L178
REVIEW	Main Review

DESCRIPTION

`UniswapV4Adapter.placeLiquidity` validates fee-module locker data only when `context.feeModule.module != address(0)`. When `context.feeModule.module == address(0)`, launcher-provided `lockerData` is forwarded to the downstream locker as-is, without checking its schema or contents.

A launcher can therefore select the no-module/v4-parity fee path while still providing `SCHEMA_LOCKER_FEE_MODULE_V1`-shaped `lockerData`. Downstream locker decodes that data independently and, if the embedded `feeModule` is nonzero, stores it as the token's fee route. Later LP-fee collection transfers fees to that embedded module and calls `receiveFees`.

This bypasses v5 fee-module validation entirely and allows an arbitrary, unregistered contract to become the LP-fee sink for the Uniswap position. The launch appears to have no runtime fee module at the v5 level, but locker-collected fees are routed through attacker-controlled module.

RECOMMENDATION

Reject `SCHEMA_LOCKER_FEE_MODULE_V1` locker data whenever `context.feeModule.module == address(0)`, and keep requiring embedded fee-module locker data to match the current v5 fee-module context when a module is configured

PROOF OF CONCEPT <https://github.com/GuardianOrg/contracts-team1-1782421575052/pull/9>

RESOLUTION

Status: Resolved.

HIGH **H-13**

PARTIALLY RESOLVED

collectRewardsWithoutUnlock Lets Fees Stolen

CATEGORY Access Control

LOCATION <https://github.com/GuardianOrg/contracts-team1-1782421575052/blob/ef734f58e018a0eee57d78d3f5a0d8f01c082edc/src/lp-lockers/ClankerLpLockerFeeConversion.sol#L457>

REVIEW Main Review

DESCRIPTION

`ClankerLpLockerFeeConversion.collectRewardsWithoutUnlock` is public even though it is intended for the hook-controlled path while the pool is already unlocked. An attacker can directly call `PoolManager.unlock`, then execute a TAKE action first on `PositionManager` via `PositionManager.modifyLiquiditiesWithoutUnlock` and transfer fee tokens to themselves. This creates negative delta debt against the shared `PositionManager` account.

Then, before the unlock ends, the attacker executes `collectRewardsWithoutUnlock(token)`. The locker then collects pending LP fees with a zero-liquidity decrease. Since the locker owns the LP NFT, the `PositionManager` ownership check passes, and the collected LP fees credit the same `PositionManager` delta account. Those credits net against the attacker-created debt, so the unlock settles while the attacker keeps the taken tokens.

As a result, pending LP fees can be stolen before normal fee collection via `collectRewardsWithoutUnlock` route.

RECOMMENDATION

Restrict `collectRewardsWithoutUnlock` to only trusted hooks or another approved collection context.

PROOF OF CONCEPT <https://github.com/GuardianOrg/contracts-team1-1782421575052/pull/10>

RESOLUTION

Status: Partially resolved.

MEDIUM **M-01**

RESOLVED

Venueld Collision With Different Pancake Fees

CATEGORY	Logical Error
LOCATION	https://github.com/GuardianOrg/contracts-team1-1782421575052/blob/ef734f58e018a0eee57d78d3f5a0d8f01c082edc/src/v5/launchers/PancakeInfinityCLClankerLauncher.sol#L170
REVIEW	Main Review

DESCRIPTION

Pancake Infinity CL launches create a pool key that includes the configured `lpFee`, but the returned Clanker `venueId` is derived only from `currency0`, `currency1`, and `parameters`. As a result, two Pancake pools for the same token pair and tick spacing but different fee tiers resolve to the same Clanker venue even though they are distinct AMM pools.

V5 supports appending additional protocol-tracked liquidity positions after launch, and those positions are not required to use the same Pancake pool as the original launch. When a token admin adds another Pancake position for the same pair and tick spacing but a different fee tier, both positions are routed through the same `launchId + venueId` fee policy. Their collected LP fees, threshold accounting, pending balances, and fee events are merged as if they came from one venue.

RECOMMENDATION

Include `poolKey.fee` in the Pancake `venueId` derivation, preferably by hashing the full canonical Pancake pool key.

RESOLUTION

Status: Resolved.

MEDIUM **M-02**

PARTIALLY RESOLVED

DevBuy ETH Can Be Stranded In UniversalRouter

CATEGORY Logical Error
LOCATION ClankerUniv4EthDevBuyV5.sol
REVIEW Main Review

DESCRIPTION

The in-scope `ClankerUniv4EthDevBuyV5` extension is a wrapper that forwards dev-buy execution to the legacy `ClankerUniv4EthDevBuy` extension. That legacy extension forwards the full native `amountIn` to `UniversalRouter` when the configured v4 hop uses native ETH.

Although the route uses `SWAP_EXACT_IN_SINGLE`, Uniswap v4 does not guarantee that the full specified input is consumed. If available liquidity is insufficient, the swap can stop before consuming the full `amountIn`. `SETTLE_ALL` then settles only the pool's actual debt, not the full ETH value sent to the router.

The extension does not sweep or refund the remaining ETH. As a result, leftover ETH stays on the shared Universal Router and can be taken by any caller through the permissionless `SWEEP` command. Additionally, ERC20 inputs can remain unused when less than the approved amount is consumed, but those leftovers stay in the extension rather than becoming permissionlessly sweepable from the router.

RECOMMENDATION

Track actual input consumed and explicitly recover any unspent input after each router swap, using a router `SWEEP` for native ETH and refunding any remaining ERC20 balance from the extension to the intended recipient.

RESOLUTION

Status: Partially resolved.

MEDIUM **M-03**

RESOLVED

Admin Rotation Is Not Recorded

CATEGORY	Logical Error
LOCATION	https://github.com/GuardianOrg/contracts-team1-1782421575052/blob/ef734f58e018a0eee57d78d3f5a0d8f01c082edc/src/v5/ClankerV5.sol#L952
REVIEW	Main Review

DESCRIPTION

Clanker v5 records both `tokenAdmin` and `originalTokenAdmin` at launch finalization.

`originalTokenAdmin` is stored as the launch-time original admin, while `tokenAdmin` is used by v5 for admin-gated lifecycle actions such as supplemental liquidity additions and liquidity migration authorization.

However, `tokenAdmin` is also immutable in the v5 deployment record. `ClankerToken.updateAdmin` can legitimately rotate the token-level admin, and B20 admin roles can be transferred or revoked, but v5 never updates the recorded `tokenAdmin`. As a result, admin-gated v5 actions continue to authorize the launch-time recorded address rather than the token's current admin.

This can DoS supplemental liquidity additions and liquidity migrations when the recorded admin is no longer available. More importantly, it can also let the old admin continue performing v5 admin-gated actions after token-level authority has been rotated away, bypassing the intended access-control change.

RECOMMENDATION

Add a v5-side admin-rotation flow for deployment records and document token level updates require record update as well.

RESOLUTION

Status: Resolved.

MEDIUM **M-04**

RESOLVED

FDV Guard Does Not Validate Denomination Token

CATEGORY Validation

LOCATION <https://github.com/GuardianOrg/contracts-team1-1782421575052/blob/ef734f58e018a0eee57d78d3f5a0d8f01c082edc/src/v5/libraries/LaunchValuationValidation.sol#L61-L74>

REVIEW Main Review

DESCRIPTION

The protocol admins can set valuation profiles via `setValuationProfile` which serves as a protocol price safety policy for a launch. This stores the `minFdvPaired` and `maxFdvPaired` values, but the FDV denomination used to enforce them is creator-controlled. The min/max values are checked against the `fdvPaired` value if a valuation guard is enabled by the creator. However, `PancakeInfinityCLAdapter.quoteLaunchFdvPaired` returns FDV denominated in `config.pairedToken`, which is supplied by the creator and is only checked to be nonzero and different from the launch token.

```
(uint256 fdvPaired, address denominationToken) =  
IClankerAmmValuationAdapter(allocation.adapter).quoteLaunchFdvPaired(  
context,  
allocation,  
liquidityTokenAmounts[i],  
allocation.adapterData  
);  
if (policy.maxFdvPaired > 0 && fdvPaired > policy.maxFdvPaired) {  
revert LaunchFdvAboveMaximum(fdvPaired, policy.maxFdvPaired);  
}
```

In the adapter, the fdv quote is calculated from the pool price and returned with the following:

```
fdvPaired = LaunchValuationMath.fdvPairedFromSqrtPriceX96(  
context.supplyAllocation.totalLaunchSupply,  
sqrtPriceX96,  
token0IsClanker  
);  
denominationToken = config.poolConfig.pairedToken;
```

M-04 FDV Guard Does Not Validate Denomination Token

CONTINUED

So the creator controls `config.poolConfig.pairedToken`, while the policy compares raw `fdvPaired` against raw `maxFdvPaired` without validating the denomination token. For example, an admin can set `maxFdvPaired = 50e18` (i.e., for weth denominated cap), but a user can pair their launch with a 6-decimal token, which will bypass the guard since the same raw `maxFdvPaired` value is used to represent a much larger amount of the paired token. As a result, a creator can satisfy an admin-governed `maxFdvPaired` while making the cap represent a far larger amount of the selected paired token than the admin intended.

RECOMMENDATION

Bind each valuation profile to an expected denomination token and decimal scale, then require the adapter's returned `denominationToken` to match before applying `minFdvPaired` or `maxFdvPaired`. Alternatively, restrict guarded launches to approved denomination tokens and normalize FDV values before comparison.

```
struct ValuationProfile {  
    bool enabled;  
    uint256 minFdvPaired;  
    uint256 maxFdvPaired;  
    address expectedDenominationToken; //@audit check against expected denom token  
    uint8 expectedDecimals; //@audit check against expected decimals  
}
```

RESOLUTION

Status: Resolved.

MEDIUM **M-05**

RESOLVED

Raw Fee Units Skew Thresholds

CATEGORY	Logical Error
LOCATION	https://github.com/GuardianOrg/contracts-team1-1782421575052/blob/ef734f58e018a0eee57d78d3f5a0d8f01c082edc/src/v5/fees/ClankerVolumeThresholdFee.sol#L376
REVIEW	Main Review

DESCRIPTION

`ClankerVolumeThresholdFee` tracks one `cumulativeRevenue` value per launch venue, but active AMM fees can be received in both pool tokens. Pancake collects and routes both `currency0` and `currency1` LP fees to the fee module, and Uniswap LP fee routing can also forward both reward tokens. Each received amount is then added directly to the same venue-level revenue counter without recording the token denomination or normalizing decimals.

As a result, fees from assets with different decimals or values are mixed as raw token units. For example, in a Pancake pool with an 18-decimal launch token and 6-decimal USDC pair, 1e18 launch-token units and 1e6 USDC units are both added directly to `cumulativeRevenue`. The USDC side is therefore effectively treated as dust relative to the launch token side, even though it may represent the economically meaningful fee asset.

This makes the volume threshold inaccurate for common mixed-decimal pools and can cause the protocol-share transition to occur too early or too late depending on which token side produces fees. The configured threshold therefore does not reliably represent fee volume or value across the venue.

Additionally, even when both assets use 18 decimals, the same raw-unit accounting remains incorrect: for example, 1e18 WETH fee units and 1e18 launch-token fee units are treated as equal revenue even though their economic values can differ by orders of magnitude.

RECOMMENDATION

Track threshold revenue in a single normalized denomination, or maintain separate per-token thresholds/accounting so fees from different assets are not summed as raw units.

RESOLUTION

Status: Resolved.

MEDIUM **M-06**

RESOLVED

Lost LP Fees When v4-parity With Pancake

CATEGORY Configuration

LOCATION <https://github.com/GuardianOrg/contracts-team1-1782421575052/blob/ef734f58e018a0eee57d78d3f5a0d8f01c082edc/src/v5/adapters/PancakeInfinityCLAdapter.sol#L131>

REVIEW Main Review

DESCRIPTION

Pancake Infinity CL allocations can currently be launched with the v4-parity fee-routing path, even though that path does not configure a runtime fee module for Pancake LP fee collection. In this path, `FeePolicyLibrary.decodeFeePolicy` returns no runtime fee module, so `PancakeInfinityCLAdapter` never calls `factoryConfigurePancakeFeeModule` for the minted Pancake LP NFT.

This leaves the Pancake launcher holding the LP NFT without a `_feeModuleRouting` entry for its `tokenId`. Pancake LP fees will accrue on the position, but the documented collection path, `collectLpFeesToFeeModule`, reverts with `FeeModuleNotConfigured`. Therefore, Pancake launches using the allowed v4-parity path lose access to their LP fees through the intended v5 fee-routing mechanism.

RECOMMENDATION

Enforce adapter-aware fee-routing compatibility so Pancake launches either configure a runtime fee module before liquidity placement or use a dedicated no-module Pancake fee collection path with explicit fee recipients.

RESOLUTION

Status: Resolved.

MEDIUM **M-07**

RESOLVED

Supplemental Pools Can Break Fees And Swaps

CATEGORY Validation

LOCATION <https://github.com/GuardianOrg/contracts-team1-1782421575052/blob/ef734f58e018a0eee57d78d3f5a0d8f01c082edc/src/v5/adapters/UniswapV4Adapter.sol#L199>, <https://github.com/GuardianOrg/contracts-team1-1782421575052/blob/ef734f58e018a0eee57d78d3f5a0d8f01c082edc/src/v5/fees/ClankerVolumeThresholdFee.sol#L111-L113>

REVIEW Main Review

DESCRIPTION

Uniswap v4 modular fee policy is stored per `(launchId, venueId)`, and all Uniswap v4 positions use the same `VENUE_ID_UNISWAP_V4`. A fixed `feeToken` policy is valid for the original pool because `ClankerVolumeThresholdFee` accepts both the launch token and the configured fee token. For example, a WETH-paired launch with `feeToken = WETH` can account launch-token fees and WETH fees.

However, when the token admin later adds a supplemental Uniswap v4 pool with a different paired token, such as USDC, the new pool is still bound to the same launch-wide Uniswap fee policy. The second `initializeFeePolicy` call is a no-op because the venue state is already initialized, so the supplemental pool's USDC fee asset is never validated against or added to the accepted fee-token set.

After USDC fees accrue, a later swap on the supplemental pool triggers the hook fee-claim path. The hook forwards USDC to `receiveFees(launchId, VENUE_ID_UNISWAP_V4, USDC, ...)`, but the fee module rejects USDC because it is neither the launch token nor the configured WETH fee token. The rejection bubbles up through the hook and reverts the swap, so trading on the supplemental pool can stop once incompatible fees are pending.

Noting that the supplemental Uniswap liquidity is currently blocked by separate pool/locker issues, but this becomes reachable once those blockers are fixed.

RECOMMENDATION

Validate supplemental pool fee assets against the already-initialized venue fee policy before binding hooks/lockers to the fee module. This prevents incompatible paired-token pools from being added through the supplemental path when the existing policy uses a fixed `feeToken`.

If supplemental pools with different paired assets are intended to be supported, scope fee policy by a pool/position identifier instead of only `(launchId, venueId)`, or document that creators must use `feeToken = address(0)` when they want one Uniswap fee policy to accept multiple paired assets.

M-07 Supplemental Pools Can Break Fees And Swaps

CONTINUED

— RESOLUTION

Status: Resolved.

MEDIUM **M-08**

PARTIALLY RESOLVED

Pancake Residuals Bypass Consumption Check

CATEGORY Rounding

LOCATION <https://github.com/GuardianOrg/contracts-team1-1782421575052/blob/ef734f58e018a0eee57d78d3f5a0d8f01c082edc/src/v5/launchers/PancakeInfinityCLClankerLauncher.sol#L198-L204>, <https://github.com/GuardianOrg/contracts-team1-1782421575052/blob/ef734f58e018a0eee57d78d3f5a0d8f01c082edc/src/v5/launchers/PancakeInfinityCLClankerLauncher.sol#L228>

REVIEW Main Review

DESCRIPTION

`PancakeInfinityCLClankerLauncher` prefunds the Pancake `CL_POSITION_MANAGER` with the full assigned `tokenAmount`, computes integer CL liquidity from that amount, and then records the original `tokenAmount` as placed liquidity. The surrounding adapter/launcher checks show the intended invariant is full consumption: if the assigned launch tokens are not consumed, the flow should revert.

However, the post-mint balance checks only inspect the launcher/adapter balances. They do not account for the residual ERC20 balances left on `CL_POSITION_MANAGER`. For normal/default Pancake ranges this appears to leave no meaningful remainder, but with extreme custom tick ranges, floor-rounded liquidity can require materially less than the pre-funded amount. The launch can finalize while v5 records the full allocation as locked LP even though a large remainder remains as raw ERC20 balance on `CL_POSITION_MANAGER`.

Because Pancake's position manager exposes a public `SWEEP` action, any caller can sweep the residual launch tokens. This can make a finalized launch appear fully collateralized by LP while part of the allocation was never actually deposited into the position.

RECOMMENDATION

Sweep residual launch tokens from `CL_POSITION_MANAGER` after minting. If the protocol strictly requires the full allocation to be added as liquidity, revert when any residual remains; however, this may DoS legitimate ranges due to dust-level rounding residue. Otherwise, enforce a bounded dust threshold and revert only when the swept residual exceeds it.

RESOLUTION

Status: Partially resolved.

LOW **L-01**

RESOLVED

setCore Missing Access Control

CATEGORY Access Control

LOCATION <https://github.com/GuardianOrg/contracts-team1-1782421575052/blob/ef734f58e018a0eee57d78d3f5a0d8f01c082edc/src/v5/ClankerDeploymentRegistry.sol#L33>

REVIEW Main Review

DESCRIPTION

`ClankerDeploymentRegistry.setCore` sets the CORE address with no caller authentication:

```
function setCore(address core_) external {  
    if (CORE != address(0) || core_ == address(0)) {  
        revert UnauthorizedCore();  
    }  
    CORE = core_;  
}
```

The deploy script constructs the registry with `address(0)` then calls `setCore` in a later transaction. Between those two transactions an attacker can call `setCore` with a malicious core address, making the deployer's `setCore(core)` revert and forcing redeployment (unless unnoticed).

RECOMMENDATION

Add access control to `setCore` (e.g. `onlyOwner` or a stored deployer address)

RESOLUTION

Status: Resolved.

LOW **L-02**

RESOLVED

Permissionless createToken Enables Launch Grief

CATEGORY	Access Control
LOCATION	https://github.com/GuardianOrg/contracts-team1-1782421575052/blob/ef734f58e018a0eee57d78d3f5a0d8f01c082edc/src/v5/token-sources/ClankerTokenSource.sol#L66
REVIEW	Main Review

— DESCRIPTION

The token launch deployment flow can call `ClankerTokenSource.createToken` or `BaseBTokenSource.createToken`, which have no access control, anyone can call them directly.

The deployed address is CREATE2-deterministic from the constructor args, therefore allows an attacker who knows a victim's pending parameters `tokenAdmin` and the full `tokenSourceData` to deploy the victim's exact address first via direct `createToken` calls. The victim's later `deployLaunch` then reverts at the `CREATE2` deploy step inside `createToken`.

Since Base has no public mempool, the attack relies on knowing the victim's parameters before-hand rather than a direct front-run attack by observing the mempool.

— RECOMMENDATION

Either gate `createToken` to the core factory (`msg.sender == CORE`) or include `msg.sender` in the salt.

— RESOLUTION

Status: Resolved.

LOW **L-03**

RESOLVED

Caller launchId Enables DoS And Collision

CATEGORY	DoS
LOCATION	https://github.com/GuardianOrg/contracts-team1-1782421575052/blob/ef734f58e018a0eee57d78d3f5a0d8f01c082edc/src/v5/ClankerV5.sol#L455
REVIEW	Main Review

DESCRIPTION

`launchId` is a caller-supplied bytes32 and `_reserveLaunch` marks it taken on first use and reverts `InvalidLaunchId` on any reuse:

```
function _reserveLaunch(bytes32 launchId, address creator) internal {  
    if (_launchIdTaken[launchId]) {  
        revert InvalidLaunchId();  
    }  
    _launchIdTaken[launchId] = true;  
    _launchStatuses[launchId] = LaunchLifecycleStatus.Reserved;  
    emit LaunchReserved(launchId, creator, LaunchLifecycleStatus.Reserved);  
}
```

Because the id is chosen by the caller and keyed globally rather than per creator, two launches can collide on the same value.

This happens maliciously if an attacker knows a victim's intended launchId in advance and submits it first, causing the victim's `deployLaunch` to revert. It can also happen naturally if launch tooling assigns sequential ids, since independent creators may pick the same value.

On Base there is no public mempool, so any attack depends on the victim's id being known beforehand rather than observed in flight.

RECOMMENDATION

Consider keying `keccak256(creator, launchId)` instead of the raw launchId, removing both the malicious DoS and the accidental collision regardless of what raw value a creator supplies.

RESOLUTION

Status: Resolved.

LOW **L-04**

RESOLVED

B-20 Descriptor Cannot Derive Address

CATEGORY Configuration

LOCATION <https://github.com/GuardianOrg/contracts-team1-1782421575052/blob/ef734f58e018a0eee57d78d3f5a0d8f01c082edc/src/v5/token-sources/BaseBTokenSource.sol#L126>, <https://github.com/GuardianOrg/contracts-team1-1782421575052/blob/ef734f58e018a0eee57d78d3f5a0d8f01c082edc/src/v5/token-sources/BaseBTokenSource.sol#L157-L158>

REVIEW Main Review

DESCRIPTION

`BaseBTokenSource` records the raw `config.salt` as the descriptor's `create2Salt` with `create2Deterministic == true`, but the true B-20 address derives from `(variant, source, salt)`, not the standard `keccak256(0xff, deployer, salt, initCodeHash)` scheme. `ClankerTokenSource`, by contrast, records `keccak256(tokenAdmin, salt)`. A consumer that treats the `create2Salt` field uniformly across sources, expecting standard CREATE2 derivation, cannot reconstruct the B-20 token address and will mispredict it.

RECOMMENDATION

Either store enough information in the descriptor to reconstruct the B-20 address (variant, source, and salt), or document that B-20 descriptors' `create2Salt` is the raw factory salt and that deriving the address additionally requires the source address and variant.

RESOLUTION

Status: Resolved.

LOW **L-05**

RESOLVED

Fee Token Can Mismatch Venue Assets

CATEGORY Validation

LOCATION <https://github.com/GuardianOrg/contracts-team1-1782421575052/blob/ef734f58e018a0eee57d78d3f5a0d8f01c082edc/src/v5/fees/ClankerVolumeThresholdFee.sol#L120>

REVIEW Main Review

DESCRIPTION

`ClankerVolumeThresholdFee` lets launchers configure an arbitrary non-zero `feeToken` without validating that it matches the assets from which the selected venue actually generates fees. The documented configuration is `feeToken = address(0)` for active Uniswap v4 and Pancake venues so pool currencies are accepted, and `feeToken = reserve` or zero for Baseline so the reserve token is used.

However, there is no validation regarding caller provided non-zero `feeToken`. If a launcher provides an incorrect non-zero `feeToken`, active venues can reject legitimate collected pool fees during `receiveFees`, while Baseline can track a token different from the reserve token Mercury is expected to pay. This breaks fee collection or distribution for the affected launch and may leave actual venue fees unattributed. Additionally, for Baseline venues, a launcher can configure one token as the reserve while setting a different token as the fee token, causing `distributeFees` to attribute and distribute passive balance increases of the incorrect `feeToken` if that token is later transferred to the fee module.

RECOMMENDATION

Validate `feeToken` against venue assets during fee policy initialization, or enforce venue-specific defaults by deriving the accepted fee token set from the initialized position data.

RESOLUTION

Status: Resolved.

LOW **L-06**

PARTIALLY RESOLVED

Public Adapters Spoof Launch Events

CATEGORY	Access Control
LOCATION	https://github.com/GuardianOrg/contracts-team1-1782421575052/blob/ef734f58e018a0eee57d78d3f5a0d8f01c082edc/src/v5/adapters/BaselineMercuryWrapper.sol#L75 , https://github.com/GuardianOrg/contracts-team1-1782421575052/blob/ef734f58e018a0eee57d78d3f5a0d8f01c082edc/src/v5/adapters/PancakeInfinityCLAdapter.sol#L91
REVIEW	Main Review

DESCRIPTION

`PancakeInfinityCLAdapter.placeLiquidity` and `BaselineMercuryWrapper.placeLiquidity` are externally callable and do not require `msg.sender` to be `ClankerV5`. A caller can pre-fund the adapter with tokens, provide arbitrary `LaunchContext`, `LiquidityAllocation`, and adapter data, then execute the adapter directly outside the core deploy or `addLiquidityPosition` flows.

This does not update `Clanker`'s deployment registry or current liquidity position registry, so core view functions remain unaffected. However, the direct call can still create real external protocol state, such as an unregistered `Pancake/Baseline` liquidity position or market, and the adapters emit

`LiquidityPlaced(context.launchId, ...)` using the caller-supplied `launchId`. As a result, an attacker can emit adapter-level events that appear to reference another user's launch and may confuse off-chain systems that trust adapter events without reconciling them against `ClankerV5` core events or registry state.

`Uniswap v4` is not affected in the same way because its adapter must call back into `ClankerV5` factory helpers guarded by `onlyActiveLiquidityAdapter`, which fails outside the core-controlled launch/add-liquidity flow. However, `Baseline` and `Pancake` skip those guarded callbacks when the caller supplies a zero fee module.

RECOMMENDATION

Consider restricting adapter `placeLiquidity` entrypoints to the authorized launch factory/core.

RESOLUTION

Status: Partially resolved.

LOW **L-07**

RESOLVED

Relayed Launch Extension Refund Mismatch

CATEGORY	Logical Error
LOCATION	https://github.com/GuardianOrg/contracts-team1-1782421575052/blob/ef734f58e018a0eee57d78d3f5a0d8f01c082edc/src/v5/ClankerV5.sol#L918
REVIEW	Main Review

DESCRIPTION

ClankerV5 supports relayed launches where `msg.sender` is an authorized relayer instead of the launch creator. In that path, the relayer supplies the full `msg.value` required by the launch envelope, including extension native budgets.

When a non-reverting extension failure occurs, ClankerV5 refunds the failed extension's `msgValue` to `tokenAdmin` rather than the payer. This may be acceptable for direct launches, but in relayed launches it sends the relayer-funded refund to a creator-controlled address while the launch continues.

RECOMMENDATION

Refund failed extension native value to the actual launch payer, or add an explicit signed refund recipient for relayed launches. Alternatively, document this behavior clearly if this is intended and accepted behavior.

RESOLUTION

Status: Resolved.

LOW **L-08**

RESOLVED

Passive Fees Can Be Misattributed

CATEGORY	Logical Error
LOCATION	https://github.com/GuardianOrg/contracts-team1-1782421575052/blob/ef734f58e018a0eee57d78d3f5a0d8f01c082edc/src/v5/fees/ClankerVolumeThresholdFee.sol#L358-L360
REVIEW	Main Review

DESCRIPTION

`ClankerVolumeThresholdFee` snapshots only the passive venue's `_distributableToken(state)` during initialization. For Baseline/passive venues this is intended to be the reserve or configured `feeToken`. However, `distributeFees` later accepts `state.token` as well, even though `state.token` was not necessarily snapshotted for passive accounting.

An attacker can launch token A with a passive venue whose fee token is B, causing only B to be snapshotted. If another passive venue later uses token A as its reserve or fee token, any uncustodied token A balance in the shared fee module can be distributed through the attacker's venue because `A == state.token` passes `_isAcceptedFeeToken` and its last passive balance is zero.

This requires the victim passive fee token to equal the attacker's launched token, so it is unlikely for standard Baseline reserves. Still, the passive accounting invariant is broken: passive venues should only distribute assets that were initialized and snapshotted for that passive venue.

RECOMMENDATION

For passive venues, snapshot and account every token that `distributeFees` accepts including the `state.token`, or restrict passive distribution to exactly the configured distributable fee token.

RESOLUTION

Status: Resolved.

LOW **L-09**

RESOLVED

Uniswap Adapter Underreports Capabilities

CATEGORY	Compatibility
LOCATION	https://github.com/GuardianOrg/contracts-team1-1782421575052/blob/ef734f58e018a0eee57d78d3f5a0d8f01c082edc/src/v5/adapters/UniswapV4Adapter.sol#L97-L101
REVIEW	Main Review

DESCRIPTION

`UniswapV4Adapter` does not declare `MODULE_CAP_MOVES_TOKENS`, while the Pancake and Baseline adapters do. This is inconsistent because Uniswap also participates in ERC20 movement: it approves launch tokens to `LAUNCH_FACTORY`, which then pulls them into the v4 locker/liquidity path.

The missing capability does not appear to affect current execution, since AMM adapters only require `MODULE_CAP_LP_PLACEMENT` and do not forbid token movement. However, it weakens the capability model for registry review, admin policy, and any UI/tooling that relies on module capability masks as a risk signal.

RECOMMENDATION

Add `MODULE_CAP_MOVES_TOKENS` to `UniswapV4Adapter` so its declared capabilities match its token-moving authority.

Additionally, consider also reviewing whether `MODULE_CAP_POOL_INIT` and `MODULE_CAP_MEV_CONTROL` should be declared for adapters that initialize pools or arm MEV modules, unless those omissions are intentional policy choices.

RESOLUTION

Status: Resolved.

LOW **L-10**

RESOLVED

BToken Descriptors Can Be False

CATEGORY Validation

LOCATION <https://github.com/GuardianOrg/contracts-team1-1782421575052/blob/ef734f58e018a0eee57d78d3f5a0d8f01c082edc/src/v5/token-sources/BaseBTokenSource.sol#L152-L153>

REVIEW Main Review

DESCRIPTION

`BaseBTokenSource` registers the token descriptor from the caller-provided config after `createB20` returns. However, B20 `initCalls` are executed before descriptor registration and Clanker only validates their selectors, not their arguments or final effects.

A launcher can use allowed `initCalls` to change the token's live name/symbol and to grant or revoke admin roles before Clanker records the descriptor. As a result, the registered descriptor and deployment record can show the original configured metadata and `tokenAdmin`, while the actual BToken already has different metadata or is controlled by different role holders at launch finalization.

RECOMMENDATION

After `createB20` returns, read back and validate the live BToken metadata and role/admin state before registering the descriptor, or disallow `initCalls` that can mutate descriptor-recorded fields during bootstrap.

RESOLUTION

Status: Resolved.

LOW **L-11**

RESOLVED

BToken Capabilities Are Underreported

CATEGORY Compatibility

LOCATION <https://github.com/GuardianOrg/contracts-team1-1782421575052/blob/ef734f58e018a0eee57d78d3f5a0d8f01c082edc/src/v5/token-sources/BaseBTokenSource.sol#L173-L182>

REVIEW Main Review

DESCRIPTION

`BaseBTokenSource` records BToken capability flags as supply-capped, mintable, pausable, and transfer-policy controlled. However, B20 tokens also expose mutable metadata and role-gated burn functionality that are not reflected in the descriptor.

This can cause indexers, UIs, and future compatibility checks to treat launched BTokens as less mutable or less permissioned than they actually are. The clearest mismatch is metadata mutability: B20 supports name, symbol, contract URI, and asset extra-metadata updates, but `TOKEN_CAP_MUTABLE_METADATA` is not set. B20 also supports role-gated burning, including `burn`, `burnWithMemo`, and `burnBlocked`, while `TOKEN_CAP_BURN` is not set.

RECOMMENDATION

`BaseBTokenSource` capability flags to include metadata and burn capabilities, or document that BToken descriptors intentionally do not advertise those behaviors.

RESOLUTION

Status: Resolved.

LOW **L-12**

RESOLVED

Airdrop V2 Allows Admin Root Replacement

CATEGORY Validation

LOCATION <https://github.com/GuardianOrg/contracts-team1-1782421575052/blob/ef734f58e018a0eee57d78d3f5a0d8f01c082edc/src/extensions/ClankerAirdropV2.sol#L35>

REVIEW Main Review

DESCRIPTION

`ClankerAirdropV5V2` forwards the configured airdrop admin and merkle root into `ClankerAirdropV2`, which custodies the launch's airdrop allocation. Unlike V1, `ClankerAirdropV2` does not reject a zero merkle root during initialization.

`updateMerkleRoot` allows the airdrop admin to replace the root, and its time-based overwrite guard is skipped when the current root is zero. As a result, a V2 airdrop can be created with `merkleRoot == 0`, custody the full airdrop allocation, block normal claims, and leave the admin able to install a new root at any later time. The admin can then reassign the previous allocation to an addresses of its choosing.

Even with a nonzero root, V2 also allows the admin to overwrite the root if no claims have occurred within the configured post-unlock overwrite window. This work as a recovery mechanism, but it weakens recipient guarantees because unclaimed allocations remain mutable after launch until at least one claim occurs.

RECOMMENDATION

Restore V1's zero-root rejection in `ClankerAirdropV2.receiveTokens` so V2 airdrops cannot be initialized in an indefinitely mutable zero-root state. Consider using a timelock for root replacement or clearly documenting that the airdrop admin can replace the root until the overwrite conditions are no longer satisfied.

RESOLUTION

Status: Resolved.

LOW **L-13**

RESOLVED

Passive Mode Accepted For Active Venues

CATEGORY	Validation
LOCATION	https://github.com/GuardianOrg/contracts-team1-1782421575052/blob/ef734f58e018a0eee57d78d3f5a0d8f01c082edc/src/v5/fees/ClankerVolumeThresholdFee.sol#L121
REVIEW	Main Review

DESCRIPTION

Uniswap v4 and Pancake Infinity CL currently support active modular fee collection paths. Their hook, locker, or launcher transfers collected fees to the fee module and then calls `receiveFees` or `receiveFeesForDepositor` to account them.

However, `ClankerVolumeThresholdFee` does not validate this venue constraint and accepts `passiveFeeCollection = true` for Uniswap and Pancake venues as well. Once configured this way, their normal active fee collection calls revert with `PassiveVenueDisallowsReceiveFees`, making modular fee collection unusable for that launch.

RECOMMENDATION

Validate `passiveFeeCollection` during fee policy initialization and reject passive mode for Uniswap v4 and Pancake Infinity CL unless those venues add a passive-compatible fee path.

RESOLUTION

Status: Resolved.

INFORMATIONAL **I-01**

RESOLVED

Asset Multiplier Not Shown in Capabilities

CATEGORY Informational

LOCATION <https://github.com/GuardianOrg/contracts-team1-1782421575052/blob/ef734f58e018a0eee57d78d3f5a0d8f01c082edc/src/v5/token-sources/BaseBTokenSource.sol#L172-L176>

REVIEW Main Review

DESCRIPTION

The B-20 Asset variant carries an operator-controlled rebase multiplier (`updateMultiplier`, `OPERATOR_ROLE`), but `_assetCapabilityFlags()` showcases no `rebase/multiplier` bit, so a consumer reading the descriptor gets no signal the token has a multiplier mechanism.

The multiplier does seem to affect standard ERC-20 `balanceOf()` or `totalSupply()` accounting, so the impact is limited. However, it is still a nonstandard B-20 Asset feature, and omitting it from `_assetCapabilityFlags()` leaves descriptor consumers without a clear signal that operator-controlled multiplier behavior exists.

RECOMMENDATION

Consider adding a `TOKEN_CAP_REBASING` (multiplier) flag and set it for asset-variant tokens, for completeness of the capability descriptor.

RESOLUTION

Status: Resolved.

INFORMATIONAL I-02

RESOLVED

Zero totalSupply Diverges Between Token Sources

CATEGORY Documentation

LOCATION <https://github.com/GuardianOrg/contracts-team1-1782421575052/blob/ef734f58e018a0eee57d78d3f5a0d8f01c082edc/src/v5/token-sources/ClankerTokenSource.sol#L61>, <https://github.com/GuardianOrg/contracts-team1-1782421575052/blob/ef734f58e018a0eee57d78d3f5a0d8f01c082edc/src/v5/token-sources/BaseBTokenSource.sol#L89>

REVIEW Main Review

DESCRIPTION

`ClankerTokenSource` silently defaults `totalSupply == 0` to the `100B default supply`, while `BaseBTokenSource` reverts `ZeroTotalSupply` on the same input. An integration that passes 0 to either source therefore gets inconsistent behavior: a 100B-supply token from one and a revert from the other. This inconsistency can cause integrations that use `totalSupply = 0` as a sentinel for using the default supply to behave differently depending on the selected token source.

RECOMMENDATION

Consider documenting this behavior or enforce consistent `totalSupply = 0` handling for both token sources.

RESOLUTION

Status: Resolved.

INFORMATIONAL I-03

RESOLVED

Baseline Launches Require Zero Extensions

CATEGORY	Validation
LOCATION	https://github.com/GuardianOrg/contracts-team1-1782421575052/blob/ef734f58e018a0eee57d78d3f5a0d8f01c082edc/src/v5/launchers/BaselineMercuryClankerLauncher.sol#L74
REVIEW	Main Review

DESCRIPTION

Clanker v5's general allocation model allows launch supply to be split between liquidity allocations and extension allocations. That works for AMM paths that can accept a partial launch-supply liquidity allocation, but it does not compose with the current Baseline Mercury path.

Baseline Mercury ultimately calls `createPoolFromInvariant`, which requires the supplied `initialPoolBTokens` to equal the token's live `totalSupply` (Reference: <https://github.com/OxBaseline/mercury/blob/6bc32cfbf4e7e2b0a09f1de814007af3c419203a/src/components/BFactory.sol#L279-L283>). Therefore, any Baseline launch with nonzero extension allocations will provide less than the full token supply to Mercury and revert late during the Baseline handoff.

This constraint is not documented in the v5 Baseline integration docs, and there is no Clanker-side validation to fail fast. As a result, otherwise well-formed Baseline launch envelopes can pass Clanker validation and only fail much later at the Mercury call.

RECOMMENDATION

Document that Baseline launches require zero extension allocations and add an early Clanker-side validation for Baseline allocations to fail before reaching Mercury

RESOLUTION

Status: Resolved.

INFORMATIONAL **I-04**

RESOLVED

Warning Regarding B20 Transfer Policies

CATEGORY	Trust Assumptions
LOCATION	https://github.com/GuardianOrg/contracts-team1-1782421575052/blob/ef734f58e018a0eee57d78d3f5a0d8f01c082edc/src/v5/token-sources/BaseBTokenSource.sol#L175
REVIEW	Main Review

DESCRIPTION

B20 tokens launched through Clanker may configure transfer policies during `initCalls`, or admin can update policies after launc. These policies can restrict which addresses are allowed to send tokens, receive tokens, or execute `transferFrom`.

A restrictive policy can be configured at launch to allow the Clanker deployment flow and AMM setup while limiting normal user transfers afterward. For example, a malicious allowlisted sender policy can permit protocol setup addresses and pool-side transfers, allowing users to buy tokens, while preventing ordinary users from sending tokens back to the pool to sell. This means a B20 token can be launched through Clanker with honeypot-like transfer behavior from the beginning.

RECOMMENDATION

Document this behavior clearly and warn users for B20 policies.

RESOLUTION

Status: Resolved.

INFORMATIONAL I-05

RESOLVED

Misleading InvalidLockerRewards Error Usage

CATEGORY	Error
LOCATION	https://github.com/GuardianOrg/contracts-team1-1782421575052/blob/ef734f58e018a0eee57d78d3f5a0d8f01c082edc/src/v5/adapters/UniswapV4Adapter.sol#L382-L390
REVIEW	Main Review

DESCRIPTION

`UniswapV4Adapter._validateConfig` validates both reward configuration and liquidity position configuration. However, failures in the position array checks for `tickLower`, `tickUpper`, and `positionBps` also revert with `InvalidLockerRewards`, which is misleading as these fields define LP position ranges and allocation splits, not reward configuration.

RECOMMENDATION

Use a separate error such as `InvalidLockerPositions` for tick range and `positionBps` validation failures.

RESOLUTION

Status: Resolved.

INFORMATIONAL I-06

RESOLVED

Supplemental MEV Reinitialization Risk

CATEGORY	Warning
LOCATION	https://github.com/GuardianOrg/contracts-team1-1782421575052/blob/ef734f58e018a0eee57d78d3f5a0d8f01c082edc/src/v5/adapters/UniswapV4Adapter.sol#L313
REVIEW	Main Review

DESCRIPTION

Current Uniswap supplemental liquidity additions is already blocked by separate pool-initialization and locker-state issues (H-07, H-11). However, even if those blockers are fixed, same-pool supplemental liquidity may still fail because the v5 supplemental lifecycle always calls `finalizeSupplementalPosition` after placement, and the Uniswap adapter always attempts to initialize the MEV module for the position's pool.

For pools whose MEV module was already initialized during the original launch, this can cause secondary failures or unexpected behavior. One-time MEV modules such as descending-fee or sniper-auction modules revert on a second initialization, while delay-based modules may overwrite the unlock time and re-enable/re-lock the pool. This means a future fix for supplemental liquidity should also account for MEV state, not only pool and locker state.

RECOMMENDATION

When implementing dedicated supplemental liquidity flows, distinguish fresh-pool additions from same-pool additions and only initialize MEV for pools that have not already had their MEV module initialized

RESOLUTION

Status: Resolved.

INFORMATIONAL **I-07**

RESOLVED

Pancake Allows Arbitrary Static LP Fee

CATEGORY Validation

LOCATION <https://github.com/GuardianOrg/contracts-team1-1782421575052/blob/ef734f58e018a0eee57d78d3f5a0d8f01c082edc/src/v5/libraries/UniswapV4PoolKeyValidation.sol#L37>,
<https://github.com/GuardianOrg/contracts-team1-1782421575052/blob/ef734f58e018a0eee57d78d3f5a0d8f01c082edc/src/v5/launchers/libraries/PancakeInfinityLaunchParams.sol#L13>

REVIEW Main Review

DESCRIPTION

Uniswap pools are forced to use `fee == DYNAMIC_FEE_FLAG` through `UniswapV4PoolKeyValidation`, making fees hook-managed capped at `MAX_LP_FEE`, where creators select a hook contract from a protocol supplied set. The Pancake path instead accepts creator-supplied `params.lpFee` and creates the pool with `hooks = address(0)`, with no validation on the static fee.

Buyers are still able to see this when interacting with the pool, this creates inconsistent behavior between Pancake and Uniswap launches.

RECOMMENDATION

Bound `params.lpFee` to an approved range, or force a fee path similar to Uniswap. If arbitrary static Pancake fees are intended, document the asymmetry for users and integrators.

RESOLUTION

Status: Resolved.

INFORMATIONAL I-08

RESOLVED

Baseline Fee Sink Remains Mutable

CATEGORY	Warning
LOCATION	https://github.com/GuardianOrg/contracts-team1-1782421575052/blob/ef734f58e018a0eee57d78d3f5a0d8f01c082edc/src/v5/launchers/BaselineMercuryClankerLauncher.sol#L67
REVIEW	Main Review

DESCRIPTION

For modular-fee Baseline launches, `BaselineMercuryWrapper` rewrites the Mercury `feeRecipient` to the selected v5 fee module, but the separate Mercury creator field remains launcher-supplied. Because Mercury allows the recorded creator to later call `setFeeRecipient`, the initial v5 fee-module sink can be changed after launch.

This can redirect future fees that would have flowed into the v5 fee module, including amounts that the module would have split as protocol fees and beneficiary fees. This does not directly affect other launches or existing balances, and may be acceptable if launch creators are expected to control their own Baseline fee economics. However, integrators should not assume that a Baseline launch's recorded v5 fee module remains the immutable recipient of future Mercury creator fees.

RECOMMENDATION

Document this behavior or constrain the Mercury creator/fee-recipient authority for Baseline launches where immutable v5 fee-module routing is required

RESOLUTION

Status: Resolved.

INFORMATIONAL I-09

RESOLVED

Mercury Requires Full Supply At Launch

CATEGORY	Informational
LOCATION	https://github.com/GuardianOrg/contracts-team1-1782421575052/blob/ef734f58e018a0eee57d78d3f5a0d8f01c082edc/src/v5/adapters/BaselineMercuryWrapper.sol#L22
REVIEW	Main Review

DESCRIPTION

Baseline Mercury's market creation model requires the launch token allocation to be supplied when the market is initialized, and the Clanker integration forwards the full assigned token supply to Mercury during launch. While v5 docs and tests imply supplemental liquidity is intended and supported for Baseline, Mercury itself does not expose an existing-market liquidity addition mechanism like the other AMM venues.

Specifically, `v5-liquidity-ownership-and-management.md` says Baseline can "Append positions (single-sided token path)," and `test_addLiquidityPosition_baselineAdapter_appendsPosition` asserts Baseline supplemental liquidity works through a mock launcher. In production, this expectation does not match Mercury's market model, so Baseline should be treated as a day-one allocation venue.

RECOMMENDATION

Update the docs/tests to mark Baseline Mercury as day-one allocation only and cannot support supplemental liquidity unless the downstream protocol implements it.

RESOLUTION

Status: Resolved.

INFORMATIONAL I-10

PARTIALLY RESOLVED

Supplemental Liquidity Re-binds Revoked Fee Module

CATEGORY	Informational
LOCATION	https://github.com/GuardianOrg/contracts-team1-1782421575052/blob/ef734f58e018a0eee57d78d3f5a0d8f01c082edc/src/v5/ClankerCompatibilityRegistry.sol#L214
REVIEW	Main Review

DESCRIPTION

`deployLaunch` validates the concrete runtime fee module, but `addLiquidityPosition` rebuilds `LaunchContext` from the immutable deployment record and `validateSupplementalLiquidityPath` only revalidates the token source, launch mechanic, fee-routing module, and added adapter path. If governance later revokes the recorded fee module, a token admin can still add a supplemental position which re-binds and authorizes `context.feeModule.module` for the new pool without checking that the fee module remains enabled and unrevoked.

Already-deployed positions should continue using the fee module they were launched with, even if that module is later revoked, because disabling existing fee routes can break live deployments. However, whether a newly added supplemental position under the same `launchId` should be treated as existing usage or as a new module binding should be an explicit protocol decision.

If revocation is intended to block only new launches, the current behavior is acceptable; if it is intended to block all new future fee-module bindings, supplemental liquidity should revalidate the recorded fee module before placement.

RECOMMENDATION

Define revocation semantics for supplemental liquidity. If revoked modules should not be used for new supplemental fee routes, revalidate `record.feeModule` during `addLiquidityPosition` before adapter execution.

RESOLUTION

Status: Partially resolved.

INFORMATIONAL **I-11**

RESOLVED

Pancake Default Params Ignore Decimals

CATEGORY	Informational
LOCATION	https://github.com/GuardianOrg/contracts-team1-1782421575052/blob/ef734f58e018a0eee57d78d3f5a0d8f01c082edc/src/v5/launchers/libraries/PancakeInfinityLaunchParams.sol#L42-L57
REVIEW	Main Review

DESCRIPTION

Pancake launches may omit `launchParams`, in which case the launcher falls back to protocol defaults. These defaults use fixed raw tick/price based only on token ordering. They are not adjusted for paired-token decimals, so using assets like USDC can initialize the pool at an unexpected human-normalized price.

RECOMMENDATION

Document that empty Pancake `launchParams` are raw defaults, or require explicit launch parameters when using arbitrary paired tokens.

RESOLUTION

Status: Resolved.

HIGH **H-01**

ACKNOWLEDGED

Supplemental Liquidity Can Still Be DoS'ed

CATEGORY	DoS
LOCATION	https://github.com/GuardianOrg/contracts-team1-1782421575052/blob/90655e9efbf4504766004279cfb330b1a8697645/src/v5/launchers/PancakeInfinityCLClankerLauncher.sol#L214 , https://github.com/GuardianOrg/contracts-team1-1782421575052/blob/90655e9efbf4504766004279cfb330b1a8697645/src/v5/launchers/PancakeInfinityCLClankerLauncher.sol#L270
REVIEW	Remediation Review

— DESCRIPTION

As part of the previous Pancake residual-token fix, `_mintSingleSidedLiquidity` was updated to include a Pancake `SWEEP` action after minting. This is safe during the initial launch because the launch token is unique and cannot be pre-donated before deployment, which is also reflected by the code comment: "Shared PM sweeps are safe for launch tokens (unique per deployment)".

However, initial liquidity and supplemental liquidity are now separate flows, but both paths use the same `_mintSingleSidedLiquidity` helper. After launch, the token is live and anyone can transfer even 1 wei of the launch token directly to the shared Pancake `CL_POSITION_MANAGER`. This causes unconditional SWEEP to pull donated dust back to the launcher, causing the launcher's exact balance-delta check to fail. As a result, anyone can still DoS the supplemental addition similar to the previous donation issue on main review but with a different path.

— RECOMMENDATION

Consider using separate minting logic for supplemental liquidity like performing delta based accounting, or make the sweep launch-only.

If supplemental minting also intended to have residual handling, account only for the current call's residual rather than sweeping arbitrary pre-existing balances.

PROOF OF CONCEPT <https://github.com/GuardianOrg/contracts-team1-1782421575052/pull/11>

— RESOLUTION

Status: Acknowledged.

HIGH **H-02**

ACKNOWLEDGED

Launches With ETH Buy Can Easily DoS'ed

CATEGORY	DoS
LOCATION	https://github.com/GuardianOrg/contracts-team1-1782421575052/blob/90655e9efbf4504766004279cfb330b1a8697645/src/v5/extensions/ClankerUniv4EthDevBuyV5.sol#L15 , https://github.com/GuardianOrg/contracts-team1-1782421575052/blob/90655e9efbf4504766004279cfb330b1a8697645/src/extensions/ClankerUniv4EthDevBuy.sol#L172-L174
REVIEW	Remediation Review

DESCRIPTION

The previous ETH dev-buy remediation added router sweep/refund handling for partial exact-input swaps, but also introduced a strict pre-swap check that reverts with `UnexpectedRouterEthBalance` whenever the shared Universal Router already has any native ETH balance.

The check is too strict because anyone can leave ETH dust on `Universal Router` through its payable `execute` function without performing any action, for example by calling it with empty commands and inputs. Once the router has even 1 wei of native ETH, every launch using native ETH dev-buy reverts with `UnexpectedRouterEthBalance`, causing a permissionless DoS of that launch path.

RECOMMENDATION

Remove the global router pre-balance requirement and track/refund only the ETH delta created by the current dev-buy operation, without depending on the router's unrelated pre-existing balance

PROOF OF CONCEPT <https://github.com/GuardianOrg/contracts-team1-1782421575052/pull/12>

RESOLUTION

Status: Acknowledged.

HIGH **H-03**

ACKNOWLEDGED

Broken Supplemental Liquidity Due To venueId

CATEGORY	DoS
LOCATION	https://github.com/GuardianOrg/contracts-team1-1782421575052/blob/90655e9efbf4504766004279cfb330b1a8697645/src/v5/launchers/PancakeInfinityCLClankerLauncher.sol#L224
REVIEW	Remediation Review

DESCRIPTION

`PancakeInfinityCLClankerLauncher` uses different venue id derivation between launch-time and supplemental liquidity placement. During initial placement, the launcher records the venue id with `PancakeVenueIdLib.venueId(poolKey)`, which hashes the full Pancake pool key including `currency0`, `currency1`, `hooks`, `poolManager`, `fee`, and `parameters`.

However, during `placeInfinityCLSupplementalLiquidity`, after validating that the supplemental request targets the same pool key, it returns a venue id derived only from `currency0`, `currency1`, and `parameters`. `PancakeInfinityCLAdapter.placeSupplementalLiquidity` then compares that returned venue id against the recorded launch position's venue id and reverts with `SupplementalLiquidityPoolMismatch`.

As a result, Pancake same-pool supplemental liquidity is unusable. This is not an intentionally rejected new-pool case: the supplemental call targets the same pool, but fails because the launcher returns a non-canonical venue id. Token admins cannot append additional Pancake liquidity to their launch pool through the intended v5 supplemental liquidity flow.

Additionally, this is separate from the prior Pancake supplemental-liquidity issue (remediations H-01) as they have different root causes: that one is caused by sweep/balance accounting, while this one is caused by inconsistent `venueId` derivation, so fixing one does not resolve the other failure path.

RECOMMENDATION

Use the same canonical `PancakeVenueIdLib.venueId(poolKey)` derivation in both initial and supplemental Pancake liquidity paths.

PROOF OF CONCEPT <https://github.com/GuardianOrg/contracts-team1-1782421575052/pull/13>

RESOLUTION

Status: Acknowledged.

HIGH **H-04**

ACKNOWLEDGED

Min FDV Bypass Via 1 BPS Allocation

CATEGORY	Logical Error
LOCATION	https://github.com/GuardianOrg/contracts-team1-1782421575052/blob/90655e9efbf4504766004279cfb330b1a8697645/src/v5/adapters/UniswapV4Adapter.sol#L368-L376
REVIEW	Remediation Review

— DESCRIPTION

The H-05 remediation moved Uniswap valuation away from the initialized spot and toward executable liquidity, which resolves the previous max-FDV gap where liquidity could sit above the guarded price. However, the new quote logic represents all Uniswap ranges with a single value: the highest nonzero `tickLower`. While this remediation works for a maximum-FDV cap, it is not sufficient for a minimum-FDV floor.

A creator can place 9,999 bps of launch liquidity at a low tick that would fail the governed minimum FDV, then add a 1 bps decoy position at a higher tick that satisfies the floor. `quoteLaunchFdvPaired` reports only the high-tick position, so `LaunchValuationValidation` accepts the launch even though almost all executable supply is available at the prohibited low valuation.

A buyer can then acquire from the low range while the deployment appears to have passed the minimum-FDV guard. Later buyers or integrations relying on the guard are exposed to a manipulated launch where the creator/early buyer captured supply under the prohibited floor.

— RECOMMENDATION

Validate every actually funded range boundary instead of selecting one representative high tick. Enforce the minimum against the lowest executable funded boundary and the maximum against the highest.

PROOF OF CONCEPT <https://github.com/GuardianOrg/contracts-team1-1782421575052/pull/15>

— RESOLUTION

Status: Acknowledged.

HIGH **H-05**

ACKNOWLEDGED

Hook-Only Fee Collection Still Allows Stolen Fee

CATEGORY Access Control

LOCATION <https://github.com/GuardianOrg/contracts-team1-1782421575052/blob/90655e9efbf4504766004279cfb330b1a8697645/src/lp-lockers/ClankerLpLockerFeeConversion.sol#L523-L532>

REVIEW Remediation Review

DESCRIPTION

H-13 was remediated by restricting `collectRewardsWithoutUnlock` so only the token's registered pool hook can call it. This blocks the direct public-call variant where an attacker opens `PoolManager.unlock`, preloads debt against the shared `PositionManager` delta account, and then calls the locker directly. However, additional testing surfaced that a similar debt-netting attack path is still possible through a trusted hook.

An attacker can open a `PoolManager.unlock`, call `PositionManager.modifyLiquiditiesWithoutUnlock(TAKE)` to create debt on the shared `PositionManager` account, and then perform a normal swap on the victim Clanker pool within the same unlock. During `beforeSwap`, the trusted Clanker hook calls `collectRewardsWithoutUnlock`, so the hook-only caller check passes. The locker then collects LP fees through the same shared `PositionManager` delta account, and those fee credits net against the attacker's preloaded debt.

As a result, pending LP fees can still be stolen even though the locker is only called by the registered hook. Fee recipients receive only the remaining net collection, while the attacker keeps the tokens taken before the hook-triggered collection.

RECOMMENDATION

Require zero `PositionManager` transient currency deltas for both pool currencies before unlocked fee collection. If either delta is nonzero, consider skipping hook-triggered collection instead of reverting, since reverting could DoS legitimate composed transactions with preexisting deltas.

Note that this may delay fee realization in those composed transactions, but fees can still be collected later through the locked collection path.

PROOF OF CONCEPT <https://github.com/GuardianOrg/contracts-team1-1782421575052/pull/16>

H-05 Hook-Only Fee Collection Still Allows Stolen Fee

CONTINUED

— **RESOLUTION**

Status: Acknowledged.

HIGH **H-06**

ACKNOWLEDGED

Pancake Launches Revert On Rounding Dust

CATEGORY	Compatibility
LOCATION	https://github.com/GuardianOrg/contracts-team1-1782421575052/blob/90655e9efbf4504766004279cfb330b1a8697645/src/v5/launchers/PancakeInfinityCLClankerLauncher.sol#L166-L172
REVIEW	Remediation Review

— DESCRIPTION

M-08 was fixed by enforcing a full-consumption invariant after sweeping residual launch tokens, which prevents launch tokens from remaining outside the recorded LP position. However, the non-tolerant full-consumption check also makes normal Pancake CL rounding incompatible with many realistic launch ranges.

Pancake Infinity CL launch liquidity is minted by converting the requested launch-token amount into a floor-rounded liquidity value, then settling the actual token amount required for that liquidity. Because the conversion is effectively `amount -> liquidity -> amount`, the settled amount can be slightly lower than the original requested amount for otherwise valid tick ranges.

With the default 100B token supply, a \$10k-\$100k FDV launch corresponds to roughly \$0.0000001-\$0.000001 per Clanker token. At an assumed \$2,000/WETH, that maps to approximately `[-237240, -214140]` raw WETH/token ticks. With USDC as the paired token, assuming \$1/USDC and 6 decimals, the same human price band maps to approximately `[-437520, -414480]` ticks.

While the launches at default range is successful, realistic FDV-aware ranges such as \$10k-\$50k, \$10k-\$100k, \$100k-\$200k, \$100k-\$1M, \$1M-\$10M, and \$10M-\$100M leave residual dust across WETH-paired and USDC-paired launches, for both Clanker `token0` and `token1` ordering. These launches revert with `LaunchTokensNotFullyConsumed` even though the configured ranges are economically realistic and the leftover amount is only a rounding artifact.

— RECOMMENDATION

Sweep residual launch tokens and allow a tightly bounded dust tolerance instead of requiring exact full-token consumption for every valid Pancake range, and consider refunding the tolerated dust. Revert only when the residual exceeds that tolerance.

PROOF OF CONCEPT <https://github.com/GuardianOrg/contracts-team1-1782421575052/pull/17>

H-06 Pancake Launches Revert On Rounding Dust

CONTINUED

— **RESOLUTION**

Status: Acknowledged.

HIGH **H-07**

ACKNOWLEDGED

Shared Hook Enables Cross-Pool Fee Theft

CATEGORY Logical Error
LOCATION ClankerHookV2
REVIEW Remediation Review

DESCRIPTION

Hook protocol fees are stored as ERC-6909 claims keyed only by the shared hook address and currency. At the beginning of a swap, `_hookFeeClaim` burns the hook's entire balance for the current paired currency and credits it to the launch associated with whichever pool swaps next, without tracking which pool generated the fees.

An attacker can create a pool using the same hook and paired currency, wait for a confirmed victim swap to generate fees, and execute a small regular swap through the attacker pool. The victim's pending fees are credited to the attacker launch and can be distributed to attacker-controlled beneficiaries, making the theft permissionless and repeatable.

Additionally, this does not require an active attack and occurs naturally, as each swap assigns the previous swap's pending protocol fees to whichever shared-currency pool swaps next.

Note: This issue was identified in the out-of-scope legacy `ClankerHookV2` contract while testing the in-scope V5 integration; the contract was not audited as part of this engagement, and this finding should not be interpreted as a comprehensive review or security assurance of that contract.

RECOMMENDATION

Track accrued hook claims per `PoolId` and forward only the amount generated by the pool currently being claimed.

PROOF OF CONCEPT <https://github.com/GuardianOrg/contracts-team1-1782421575052/pull/18>

RESOLUTION

Status: Acknowledged.

MEDIUM **M-01**

ACKNOWLEDGED

Crosschain FDV Uses Local Supply

CATEGORY	Logical Error
LOCATION	https://github.com/GuardianOrg/contracts-team1-1782421575052/blob/90655e9efbf4504766004279cfb330b1a8697645/src/v5/token-sources/SyncedTokenSource.sol#L140 , https://github.com/GuardianOrg/contracts-team1-1782421575052/blob/90655e9efbf4504766004279cfb330b1a8697645/src/v5/token-sources/SyncedTokenSource.sol#L158
REVIEW	Remediation Review

DESCRIPTION

`SyncedTokenSource` reports only the current chain's `LOCAL_GENESIS_ALLOCATION` as `totalLaunchSupply`, while the deployed `ClankerTokenCrosschain` also has an immutable `GLOBAL_SUPPLY` and controller-gated bridge minting. Core then uses this local `totalLaunchSupply` for launch supply accounting and valuation checks.

As a result, valuation-guarded launches can approve a crosschain launch based only on the local tranche rather than the full global token supply. For example, a launch with 30% of supply minted on the current chain and 70% allocated to other chains will be valued using only the 30% local tranche, even though the remaining supply can later bridge into the same market. This can make a launch appear to satisfy max-FDV limits while the actual global FDV is materially higher.

RECOMMENDATION

For `SyncedTokenSource`, compute valuation against `GLOBAL_SUPPLY`, or disallow valuation-guarded launch paths unless the policy explicitly defines the limit as local-chain FDV only.

RESOLUTION

Status: Acknowledged.

MEDIUM **M-02**

ACKNOWLEDGED

One Token-Restricted Beneficiary Blocks Fees

CATEGORY	Logical Error
LOCATION	https://github.com/GuardianOrg/contracts-team1-1782421575052/blob/90655e9efbf4504766004279cfb330b1a8697645/src/v5/fees/ClankerVolumeThresholdFee.sol#L524
REVIEW	Remediation Review

DESCRIPTION

`ClankerVolumeThresholdFee` stores the creator-selected beneficiary list during fee policy initialization and later distributes each venue's fees through a single push-based `distributeFees` call. The function transfers the protocol share and then pushes each beneficiary share with `SafeERC20.safeTransfer`.

If the fee token rejects transfer to any configured beneficiary, the entire distribution reverts. This rolls back the protocol payment, all other beneficiary payments, and the pending-fee accounting updates.

This can happen with B20 transfer policies or externally blacklisted fee assets, and permanently lock pending and future fees for the affected launch, venue, and token.

RECOMMENDATION

Use pull-based per-recipient accounting or otherwise isolate failed payouts so one restricted beneficiary cannot block distribution for the protocol and all other recipients. Alternatively, consider adding an authorized beneficiary-rotation path for permanently unpayable recipients.

RESOLUTION

Status: Acknowledged.

MEDIUM **M-03**

ACKNOWLEDGED

Pancake Defaults Depend On Token Order

CATEGORY	Logical Error
LOCATION	https://github.com/GuardianOrg/contracts-team1-1782421575052/blob/90655e9efbf4504766004279cfb330b1a8697645/src/v5/launchers/libraries/PancakeInfinityLaunchParams.sol#L94 , https://github.com/GuardianOrg/contracts-team1-1782421575052/blob/90655e9efbf4504766004279cfb330b1a8697645/src/v5/launchers/libraries/PancakeInfinityLaunchParams.sol#L138-L144
REVIEW	Remediation Review

DESCRIPTION

`PancakeInfinityLaunchParams` builds default launch parameters from `_configuredDefaults(token0IsClanker)` and then passes the result through `_orient`. For token1-Clanker launches, `_configuredDefaults(false)` already selects the positive canonical range `[60, 6000]`, but `_orient(false)` mirrors it again into the negative pool-side range `[-6000, -60]`.

As a result, default Pancake launches start at different normalized prices depending only on token address ordering. When Clanker is `token0`, the pool starts at `tickLower = -6000`, so the launch price is `pairedToken / Clanker` at tick -6000. When Clanker is `token1`, the final pool-side range becomes `[-6000, -60]` and the pool starts at `tickUpper = -60`; because pool price is then `Clanker / pairedToken`, the normalized `pairedToken / Clanker` price is the inverse, equivalent to tick +60.

This means two otherwise identical launches using empty/default Pancake params can differ by 6060 ticks, roughly 1.83x, solely due to token sorting. The valuation quote uses the same order-dependent defaults, so it reports the divergent FDV instead of correcting it. This can change early buyer execution and make valuation-guard outcomes depend on token address order.

RECOMMENDATION

Define Pancake defaults in one canonical `pairedToken / Clanker` price frame, then apply token-order orientation exactly once for pool placement and valuation.

RESOLUTION

Status: Acknowledged.

MEDIUM **M-04**

ACKNOWLEDGED

Baseline Uses Local Supply As Full Supply

CATEGORY	Compatibility
LOCATION	https://github.com/GuardianOrg/contracts-team1-1782421575052/blob/90655e9efbf4504766004279cfb330b1a8697645/src/v5/launchers/BaselineMercuryClankerLauncher.sol#L74
REVIEW	Remediation Review

— DESCRIPTION

Baseline Mercury placement requires the full token supply to be deposited at launch. `SyncedTokenSource` only mints and hands core the chain-local genesis tranche, but at deployment time that tranche equals the token's live local `totalSupply`, so the Baseline full-supply check passes.

After the corresponding crosschain deployments and bridge mints occur, additional legitimate supply can arrive on the initial chain. Mercury and Clanker still record the launch against the smaller local tranche, creating a discrepancy between the recorded full-supply market and the token's actual crosschain supply.

— RECOMMENDATION

Consider rejecting Synced/crosschain token sources for Baseline. Alternatively, if Baseline supports a crosschain-specific initialization path, use that path explicitly so the market is initialized against global supply rather than deployment-time local supply.

— RESOLUTION

Status: Acknowledged.

MEDIUM **M-05**

ACKNOWLEDGED

Zero Protocol Numerators Disable Fees

CATEGORY	Logical Error
LOCATION	https://github.com/GuardianOrg/contracts-team1-1782421575052/blob/90655e9efbf4504766004279cfb330b1a8697645/src/v5/fees/ClankerVolumeThresholdFee.sol#L252-L253
REVIEW	Remediation Review

— DESCRIPTION

`ClankerVolumeThresholdFee` lets creators set `initialProtocolNumerator` and `postThresholdProtocolNumerator`, and validation only enforces an upper bound. The protocol recipient is governance-controlled, but the protocol share can still be set to zero.

A creator can configure both numerators as zero and route all future venue fees to creator-selected beneficiaries. They can also set `volumeThreshold` to 0 as well.

This makes protocol trading-fee revenue optional for modular-fee launches. If Clanker is expected to earn protocol fees from modular fee routes, this allows creators to opt out permanently.

— RECOMMENDATION

Enforce governance-approved fee profiles or nonzero minimum protocol numerators for both threshold states.

— RESOLUTION

Status: Acknowledged.

LOW **L-01**

ACKNOWLEDGED

Mechanic Context Uses Raw Launch Id

CATEGORY	Compatibility
LOCATION	https://github.com/GuardianOrg/contracts-team1-1782421575052/blob/90655e9efbf4504766004279cfb330b1a8697645/src/v5/mechanics/DefaultLaunchMechanic.sol#L58
REVIEW	Remediation Review

DESCRIPTION

Core now derives the canonical launch id as `keccak256(creator, envelope.launchId)` and uses it in `LaunchContext.launchId`, registry records, events, fees, and position tracking. However, the default and valuation-guarded launch mechanics still encode the raw `envelope.launchId` inside `launchMechanicContext`.

Noting that current production modules do not use that embedded id for state. Future consumers decoding mechanic context may treat the raw id as canonical and treat it incorrectly

RECOMMENDATION

Encode the canonical launch id in mechanic context too, or document the embedded value as the raw/client launch id.

RESOLUTION

Status: Acknowledged.

LOW **L-02**

ACKNOWLEDGED

Revoked Fee Module Blocks Supplemental Adds

CATEGORY	DoS
LOCATION	https://github.com/GuardianOrg/contracts-team1-1782421575052/blob/90655e9efbf4504766004279cfb330b1a8697645/src/v5/ClankerCompatibilityRegistry.sol#L278
REVIEW	Remediation Review

DESCRIPTION

`addLiquidityPosition` revalidates the launch's recorded runtime fee module before allowing supplemental liquidity. If that module is later revoked or disabled, the launch can keep using its already-bound fee route, but new supplemental liquidity reverts because rebinding the revoked module is prevented.

Ideally, a launch that originally used fee module **A** should not be able to bind **A** again after **A** is revoked, but it should have a way to use a governance-approved replacement fee module **B** for future liquidity additions.

However, that path is not supported at the moment, so the fix also blocks all future supplemental liquidity for launches that used the revoked module.

RECOMMENDATION

Evaluate whether this behavior is intended and document that revoking a fee module disables future supplemental liquidity for launches using that module.

If continued liquidity additions are required after revocation, add an explicit validated fee-module replacement path. However, this may increase the complexity and introduce accounting concerns.

RESOLUTION

Status: Acknowledged.

LOW **L-03**

ACKNOWLEDGED

Missing onlyLaunchFactory When Adding Liquidity

CATEGORY Access Control

LOCATION <https://github.com/GuardianOrg/contracts-team1-1782421575052/blob/90655e9efbf4504766004279cfb330b1a8697645/src/v5/adapters/PancakeInfinityCLAdapter.sol#L200>

REVIEW Remediation Review

DESCRIPTION

`PancakeInfinityCLAdapter.placeLiquidity` was restricted to `onlyLaunchFactory`, but the newly added `placeSupplementalLiquidity` remains public and trusts caller-supplied context/target position data. This allows direct callers to bypass core validation and mint untracked supplemental Pancake positions, including against an existing launch pool if they supply the launch token themselves.

Noting that the same boundary exists one layer lower:

`PancakeInfinityCLClankerLauncher.placeInfinityCLLiquidity` and `placeInfinityCLSupplementalLiquidity` are also public, so callers can bypass the adapter entirely and mint unmanaged launcher-custodied NFTs if they supply the tokens.

Also, similar issues exist in the Baseline paths. However, Baseline supplemental liquidity is not supported and reverts regardless of the caller, and `launchWithTokenAllocation` requires the full token supply, so direct calls are unlikely to succeed in practice.

RECOMMENDATION

Restrict Pancake supplemental placement in adapter to the launch factory as well. Additionally, consider restricting launcher functionalities to the adapter as well.

RESOLUTION

Status: Acknowledged.

LOW **L-04**

ACKNOWLEDGED

Descriptor Records Only Local Supply

CATEGORY	Logical Error
LOCATION	https://github.com/GuardianOrg/contracts-team1-1782421575052/blob/90655e9efbf4504766004279cfb330b1a8697645/src/v5/token-sources/SyncedTokenSource.sol#L158
REVIEW	Remediation Review

— DESCRIPTION

`SyncedTokenSource` registers the token descriptor with `totalLaunchSupply` set to the current chain's `LOCAL_GENESIS_ALLOCATION`. However, the deployed `ClankerTokenCrosschain` also exposes an immutable `GLOBAL_SUPPLY` and a full chain allocation manifest, which are not recorded in the descriptor.

This can mislead indexers, UIs, or integrations that rely on `TokenDescriptorRegistry` as the normalized token metadata source. For crosschain tokens, the descriptor's `totalLaunchSupply` is only the local launch tranche, not the global supply basis of the token.

— RECOMMENDATION

Extend crosschain descriptors to include global supply and controller metadata, or clearly document that `totalLaunchSupply` is chain-local for `SyncedTokenSource` and consumers must read `GLOBAL_SUPPLY` from the token.

— RESOLUTION

Status: Acknowledged.

LOW **L-05**

ACKNOWLEDGED

Synced Pool Squatting Across Chains

CATEGORY	Frontrunning
LOCATION	Global
REVIEW	Remediation Review

— DESCRIPTION

Synced tokens have deterministic same-address deployments across chains. Once one chain is launched, the token address and pool parameters may be public before the same launch is completed on another chain.

During this gap, an attacker can call public `initializePoolOpen` on the destination chain with the future token address and expected Uniswap pool parameters. The later v5 launch then deploys the token but reverts when the adapter tries to initialize the already-created pool.

Noting that this is different from mempool-dependent front-running: deployment on the first chain can reveal the token address and pool parameters, giving anyone enough information to squat the destination-chain pool during the multi-chain deployment gap

— RECOMMENDATION

Document the sequencing risk and use protected/private execution for synced deployments where this matters.

— RESOLUTION

Status: Acknowledged.

LOW **L-06**

ACKNOWLEDGED

Reauthorization Revives Stale Forwarders

CATEGORY	Logical Error
LOCATION	https://github.com/GuardianOrg/contracts-team1-1782421575052/blob/90655e9efbf4504766004279cfb330b1a8697645/src/v5/fees/ClankerVolumeThresholdFee.sol#L536
REVIEW	Remediation Review

— DESCRIPTION

`ClankerVolumeThresholdFee` tracks whether a depositor is authorized independently from the generation used to validate its delegated fee forwarders.

When the launch factory or module owner revokes a depositor through `authorizeFeeDepositor(..., false)`, the function only clears `_authorizedDepositors`. It does not advance `_feeForwarderGeneration` or otherwise invalidate forwarders previously authorized by that depositor.

While the depositor remains revoked, both the depositor and its forwarders are rejected because `receiveFeesForDepositor` first checks `_authorizedDepositors`. However, if the same depositor is subsequently reauthorized, every forwarder whose stored generation still matches the unchanged current generation automatically becomes authorized again.

— RECOMMENDATION

If this is not explicitly intended, consider advancing the depositor's forwarder generation whenever it transitions from authorized to unauthorized, requiring forwarders to be explicitly authorized again.

— RESOLUTION

Status: Acknowledged.

LOW **L-07**

ACKNOWLEDGED

Getter Hides Paired-Token Pending Fees

CATEGORY	Unexpected Behavior
LOCATION	https://github.com/GuardianOrg/contracts-team1-1782421575052/blob/90655e9efbf4504766004279cfb330b1a8697645/src/v5/fees/ClankerVolumeThresholdFee.sol#L639
REVIEW	Remediation Review

— DESCRIPTION

`venueState.pendingBalance` reports pending fees only for the threshold-denomination token. When a venue collects both launch-token and paired-token fees, paired-token fees may be pending and distributable even though the getter reports zero.

No other view function exposes `_pendingByToken` for an arbitrary token, so keepers and interfaces relying on `venueState` may overlook paired-token fees and leave them undistributed.

— RECOMMENDATION

Consider adding a view function returning the pending balance for a specified `(launchId, venueId, token)`.

— RESOLUTION

Status: Acknowledged.

INFORMATIONAL I-01

ACKNOWLEDGED

Note Regarding Uniswap Supplemental Liquidity

CATEGORY	Informational
LOCATION	https://github.com/GuardianOrg/contracts-team1-1782421575052/blob/90655e9efbf4504766004279cfb330b1a8697645/src/v5/adapters/UniswapV4Adapter.sol#L578-L580
REVIEW	Remediation Review

DESCRIPTION

Uniswap v4 supplemental liquidity cannot be added while the launch MEV module is operational.

`UniswapV4Adapter` requires a nonzero MEV module for v4 launches, and after launch finalization the hook enables that module for the pool. During this period, `ClankerHookV2._beforeAddLiquidity` rejects liquidity additions with `MevModuleEnabled`.

Noting that this does not appear to permanently break supplemental liquidity. On the current fork path, same-pool Uniswap supplemental liquidity succeeds after `MAX_MEV_MODULE_DELAY`, which is 2 minutes, or earlier if the configured MEV module disables itself through swap flow. The behavior is therefore should be treated as an operational/documentation concern.

RECOMMENDATION

Document that Uniswap v4 supplemental liquidity can only be added after the MEV protection window has ended, or explicitly surface this condition in the supplemental liquidity flow

RESOLUTION

Status: Acknowledged.

INFORMATIONAL I-02

ACKNOWLEDGED

Refunds Require Payable Callers

CATEGORY	Documentation
LOCATION	https://github.com/GuardianOrg/contracts-team1-1782421575052/blob/90655e9efbf4504766004279cfb330b1a8697645/src/v5/ClankerV5.sol#L251 , https://github.com/GuardianOrg/contracts-team1-1782421575052/blob/90655e9efbf4504766004279cfb330b1a8697645/src/v5/ClankerV5.sol#L1014
REVIEW	Remediation Review

DESCRIPTION

Refunds during deployment or failed extension paths are returned to the `msg.sender`. However, if the caller is a contract (for example during relayed launches) without a payable `receive` or `fallback`, these refund transfers revert and the launch fails.

RECOMMENDATION

Document that callers need to be able accept native tokens for refunds.

RESOLUTION

Status: Acknowledged.

INFORMATIONAL I-03

ACKNOWLEDGED

Document Dev-Buy Recipient Also Receives Refunds

CATEGORY	Documentation
LOCATION	https://github.com/GuardianOrg/contracts-team1-1782421575052/blob/90655e9efbf4504766004279cfb330b1a8697645/src/extensions/ClankerUniv4EthDevBuy.sol#L227-L238
REVIEW	Remediation Review

DESCRIPTION

`ClankerUniv4EthDevBuy` refunds unspent dev-buy input to `devBuyData.recipient` for both native ETH and ERC20 inputs. This is reasonable because the legacy extension is called by core, so `msg.sender` is not the actual payer; refunding to `msg.sender` would send funds back to core and require additional steps to reach the actual payer.

RECOMMENDATION

Document that `devBuyData.recipient` receives both bought tokens and any unspent ETH or ERC20 input refunded after router execution.

If separate refund routing is desired, add an explicit refund-recipient parameter.

Additionally, document that refund recipient needs to be able to accept native tokens for refunds.

RESOLUTION

Status: Acknowledged.

INFORMATIONAL **I-04**

ACKNOWLEDGED

Registry Admin Update Event Is Not Emitted

CATEGORY	Events
LOCATION	https://github.com/GuardianOrg/contracts-team1-1782421575052/blob/90655e9efbf4504766004279cfb330b1a8697645/src/v5/ClankerDeploymentRegistry.sol#L78
REVIEW	Remediation Review

DESCRIPTION

Both `ClankerV5` and `ClankerDeploymentRegistry` declare `DeploymentTokenAdminUpdated`, but only core emits it.

`ClankerDeploymentRegistry.updateTokenAdmin` updates `record.tokenAdmin` without emitting the registry event.

RECOMMENDATION

Either emit the event from the registry as well, or remove the registry event declaration.

RESOLUTION

Status: Acknowledged.

INFORMATIONAL I-05

ACKNOWLEDGED

Supplemental CL Ranges Must Match Price

CATEGORY	Documentation
LOCATION	https://github.com/GuardianOrg/contracts-team1-1782421575052/blob/90655e9efbf4504766004279cfb330b1a8697645/src/v5/launchers/PancakeInfinityCLClankerLauncher.sol#L203 , https://github.com/GuardianOrg/contracts-team1-1782421575052/blob/90655e9efbf4504766004279cfb330b1a8697645/src/v5/launchers/PancakeInfinityCLClankerLauncher.sol#L215
REVIEW	Remediation Review

DESCRIPTION

Supplemental liquidity paths supply only the launch token, so the added tick range must be single-sided relative to the current pool price. If the live price is inside the range, the position requires both assets and may revert or fail exact-consumption checks.

This is expected CL behavior, but should be documented for token admins/integrators.

Noting that the same constraint exists at launch time, but the launch paths perform stricter range/starting-price validation than supplemental additions.

RECOMMENDATION

Document the provided range must be valid compared to the live price/slot0 values, and consider adding fail-fast validation where the live price is available.

RESOLUTION

Status: Acknowledged.

INFORMATIONAL **I-06**

ACKNOWLEDGED

Baseline Deploy Docs Use Old Address

CATEGORY Documentation
LOCATION v5-baseline-mercury-launcher-deploy.md
REVIEW Remediation Review

DESCRIPTION

`BaselineMercuryLauncherDeployConfig` was updated with a new `PREDICTED_ADDRESS` and `INIT_CODE_HASH` for the current `BaselineMercuryClankerLauncher` bytecode. However, `baseline-mercury-launcher-deploy.md` still documents the previous allowlist address and init-code hash

RECOMMENDATION

Update the markdown deployment guide to match `BaselineMercuryLauncherDeployConfig`

RESOLUTION

Status: Acknowledged.

INFORMATIONAL **I-07**

ACKNOWLEDGED

Warning Regarding Multi Chains

CATEGORY	Warning
LOCATION	Global
REVIEW	Remediation Review

— DESCRIPTION

Current documented deployments are mainly on chains such as Base, Arbitrum, and Unichain, where classic public mempool front-running assumptions are weaker or do not directly apply. However, the crosschain test files in the latest commit include chain id 534352, which corresponds to Scroll, and Scroll exposes pending transaction / mempool-style behavior.

If protocol is expected to be deployed on chains with public mempools, deterministic launch parameters may let attackers front-run a pending launch and directly initialize the target Uniswap v4 or Pancake pool before the official launch transaction executes. This can cause the later launch to revert with an already-initialized pool.

Noting that this is only a deployment-environment warning and can be discarded if those chain ids are present only for testing.

— RECOMMENDATION

Be aware of this situation. If chains with mempools will be used, consider private RPCs to prevent frontrunning/deterministic address DoS attacks

— RESOLUTION

Status: Acknowledged.

INFORMATIONAL **I-08**

ACKNOWLEDGED

Unbound Bridge LaunchId

CATEGORY	Validation
LOCATION	CrosschainSupplyController.sol
REVIEW	Remediation Review

DESCRIPTION

`CrosschainSupplyController.bridgeOut` accepts a caller-provided `launchId`, and `bridgeIn` only authenticates the exact message through the transport adapter; neither side verifies that the `launchId` belongs to the bridged token. This can produce real burn/mint transfers with misleading launch ids in bridge events/messages.

No production `ICrosschainTransportAdapter` implementation is included in the reviewed codebase and is OOS. Only the interface and mock adapter are present, so this may be part of unfinished crosschain integration work.

RECOMMENDATION

Bind token => launchId in the controller or require adapters to enforce it when they are production ready.

RESOLUTION

Status: Acknowledged.

INFORMATIONAL **I-09**

ACKNOWLEDGED

Passive Sinks Assume Standard ERC20

CATEGORY	Informational
LOCATION	https://github.com/GuardianOrg/contracts-team1-1782421575052/blob/90655e9efbf4504766004279cfb330b1a8697645/src/v5/fees/ClankerVolumeThresholdFee.sol#L23
REVIEW	Remediation Review

— DESCRIPTION

Passive sinks flush/recover their full token balance in one transfer. Nonstandard reserve tokens with caps, blacklists, transfer fees, or balance-dependent restrictions can make those calls revert or account unexpected amounts.

— RECOMMENDATION

Document standard ERC20 reserve requirements.

— RESOLUTION

Status: Acknowledged.

INFORMATIONAL **I-10**

ACKNOWLEDGED

Misleading Comment In Fee Module

CATEGORY	Informational
LOCATION	https://github.com/GuardianOrg/contracts-team1-1782421575052/blob/90655e9efbf4504766004279cfb330b1a8697645/src/v5/fees/ClankerVolumeThresholdFee.sol#L85
REVIEW	Remediation Review

— DESCRIPTION

The ClankerVolumeThresholdFee has this code comment: "Halves hook protocol share after a cumulative revenue threshold is reached."

However, the current behavior is not halving the protocol share as both pre and post threshold numerators are launcher provided.

— RECOMMENDATION

Update the comment. If the intended behavior is indeed halving the fees, enforce it in the implementation.

— RESOLUTION

Status: Acknowledged.

§ 05

Disclaimer.

This report is not, nor should be considered, an "endorsement" or "disapproval" of any particular project or team. This report is not, nor should be considered, an indication of the economics or value of any "product" or "asset" created by any team or project that contracts Guardian to perform a security assessment. This report does not provide any warranty or guarantee regarding the absolute bug-free nature of the technology analyzed, nor do they provide any indication of the technologies proprietors, business, business model or legal compliance.

This report should not be used in any way to make decisions around investment or involvement with any particular project. This report in no way provides investment advice, nor should be leveraged as investment advice of any sort. This report represents an extensive assessing process intending to help our customers increase the quality of their code while reducing the high level of risk presented by cryptographic tokens and blockchain technology.

Blockchain technology and cryptographic assets present a high level of ongoing risk. Guardian's position is that each company and individual are responsible for their own due diligence and continuous security. Guardian's goal is to help reduce the attack vectors and the high level of variance associated with utilizing new and consistently changing technologies, and in no way claims any guarantee of security or functionality of the technology we agree to analyze.

The assessment services provided by Guardian is subject to dependencies and under continuing development. You agree that your access and/or use, including but not limited to any services, reports, and materials, will be at your sole risk on an as-is, where-is, and as-available basis.

Cryptographic tokens are emergent technologies and carry with them high levels of technical risk and uncertainty. The assessment reports could include false positives, false negatives, and other unpredictable results. The services may access, and depend upon, multiple layers of third-parties.

Notice that smart contracts deployed on the blockchain are not resistant from internal/external exploit. Notice that active smart contract owner privileges constitute an elevated impact to any smart contract's safety and security. Therefore, Guardian does not guarantee the explicit security of the audited smart contract, regardless of the verdict.

§ 06

About Guardian.

Guardian is a smart contract security firm. Every engagement is worked by two independent teams competing on the same scope, each with its own tooling and methods, so no single reviewer's blind spot survives.

\$45B+DIGITAL ASSETS UNDER
REVIEW**200+**

ENGAGEMENTS COMPLETED

500+CRITICAL & HIGH FINDINGS
REPORTED

Reach us at audits@guardianaudits.com.